Guess X - SDD

- 1 System Design Document (SDD)
  - 1.1 Guess X — Lowest Unique Bid Auction Platform
  - 1.2 Cover Page
    - 1.2.1 Document Control
    - 1.2.2 Distribution List
    - 1.2.3 Confidentiality Notice
  - 1.3 Table of Contents
  - 1.4 1. System Overview
    - 1.4.1 1.1 Design Principles
  - 1.5 2. High-Level Architecture
    - 1.5.1 2.1 Architecture Layers
    - 1.5.2 2.2 Layer Responsibilities
  - 1.6 3. Component Diagrams and Interaction Flows
    - 1.6.1 3.1 Identity Service Components
    - 1.6.2 3.2 Auction Engine Components
    - 1.6.3 3.3 Query Service Components
    - 1.6.4 3.4 Key Interaction Flows
    - 1.6.5 3.5 State Diagrams
  - 1.7 4. Deployment Architecture
    - 1.7.1 4.1 Development Environment (Docker Compose)
    - 1.7.2 4.2 Staging Environment
    - 1.7.3 4.3 Production Environment (Kubernetes)
    - 1.7.4 4.4 Kubernetes Manifests
    - 1.7.5 4.5 Disaster Recovery and Backup
    - 1.7.6 4.6 Monitoring and Alerting
    - 1.7.7 4.7 CI/CD Pipeline
  - 1.8 5. Data Architecture
    - 1.8.1 5.1 Database Strategy
    - 1.8.2 5.2 Citus Distribution Strategy
    - 1.8.3 5.3 Redis Usage
  - 1.9 6. Scalability and Performance
    - 1.9.1 6.1 Horizontal Scaling
    - 1.9.2 6.2 Performance Optimizations
    - 1.9.3 6.3 Load Testing
  - 1.10 7. Security Architecture

        - 1.10.1 7.1 Authentication and Authorization
        - 1.10.2 7.2 Threat Mitigations
        - 1.10.3 7.3 Security Headers (Production)
    - 1.11 8. Technology Stack
        - 1.11.1 8.1 Backend Services
        - 1.11.2 8.2 Frontend Applications
        - 1.11.3 8.3 Infrastructure
        - 1.11.4 8.4 External Services
        - 1.11.5 8.5 Data Layer
    - 1.12 9. Redis High Availability
        - 1.12.1 9.1 Architecture Overview
        - 1.12.2 9.2 Configuration Detection Logic
        - 1.12.3 9.3 Sentinel Mode (Production Recommended)
        - 1.12.4 9.4 Cluster Mode (High Scale)
        - 1.12.5 9.5 Kubernetes Deployment
    - 1.13 10. Comprehensive Audit Logging
        - 1.13.1 10.1 Audit Log Entity
        - 1.13.2 10.2 Audit Service Architecture
        - 1.13.3 10.3 Audited Events by Service
        - 1.13.4 10.4 Immutability Guarantee
    - 1.14 11. Backup and Disaster Recovery
        - 1.14.1 11.1 Backup Strategy Comparison
        - 1.14.2 11.2 pgBackRest Configuration
        - 1.14.3 11.3 WAL-G Configuration
        - 1.14.4 11.4 Disaster Recovery Procedures
        - 1.14.5 11.5 Testing and Validation
    - 1.15 12. WebSocket Horizontal Scaling
        - 1.15.1 12.1 Architecture
        - 1.15.2 12.2 Implementation
        - 1.15.3 12.3 Scaling Behavior
        - 1.15.4 12.4 Kubernetes Deployment
    - 1.16 13. Architecture Decision Records (ADR)
        - 1.16.1 ADR-001: Microservices over Modular Monolith
        - 1.16.2 ADR-002: Prisma ORM with Raw SQL Migrations
        - 1.16.3 ADR-003: Better-auth for Identity, JwtAuthGuard for Downstream Services
        - 1.16.4 ADR-004: Redis as Cache, Coordination, and Real-Time Backbone
        - 1.16.5 ADR-005: Citus-First Distributed PostgreSQL
        - 1.16.6 ADR-006: AES-256-GCM Bid Encryption at Rest
        - 1.16.7 ADR-007: Immutable Audit Logging

- 1.17 14. Capacity Planning
  - 1.17.1 14.1 Expected Load Profile
  - 1.17.2 14.2 Resource Requirements
  - 1.17.3 14.3 Scaling Thresholds
  - 1.17.4 14.4 Growth Headroom
- 1.18 15. Security Architecture Details
  - 1.18.1 15.1 Threat Model (STRIDE)
  - 1.18.2 15.2 Attack Surface Analysis
  - 1.18.3 15.3 Defense in Depth
- 1.19 16. Operational Architecture
  - 1.19.1 16.1 Deployment Pipeline
  - 1.19.2 16.2 Monitoring
  - 1.19.3 16.3 Alerting
  - 1.19.4 16.4 Incident Response
- 1.20 17. Data Governance
  - 1.20.1 17.1 Data Classification
  - 1.20.2 17.2 Data Lifecycle
  - 1.20.3 17.3 Retention and Disposal
  - 1.20.4 17.4 Privacy Controls
- 1.21 18. Integration Architecture
  - 1.21.1 18.1 API Gateway
  - 1.21.2 18.2 Service Mesh & Internal Integration
  - 1.21.3 18.3 Event-Driven Patterns
  - 1.21.4 18.4 External Integrations
- 1.22 19. Feature Architecture Deep Dive
  - 1.22.1 19.1 Redis Caching Architecture
  - 1.22.2 19.2 Product Approval Workflow
  - 1.22.3 19.3 Settlement Report Data Flow
  - 1.22.4 19.4 SMS Reminder System Architecture
  - 1.22.5 19.5 Terms & Conditions Acceptance Flow
  - 1.22.6 19.6 Advanced Analytics Dashboard Architecture
- 1.23 20. Performance Benchmarks
  - 1.23.1 20.1 API Endpoint Benchmarks (k6, staging)
  - 1.23.2 20.2 System-Level Benchmarks
  - 1.23.3 20.3 Load Test Scenarios
- 1.24 21. Production Deployment Topology
- 1.25 22. Disaster Recovery Runbook Summary
  - 1.25.1 22.1 Runbook Index
  - 1.25.2 22.2 RB-01 Full Region Failover (Summary)
  - 1.25.3 22.3 RB-02 Database Failover (Summary)

- 1.25.4 22.4 RB-04 Point-in-Time Recovery (Summary)
- 1.26 23. Security Hardening Checklist
  - 1.26.1 23.1 Infrastructure
  - 1.26.2 23.2 Application
  - 1.26.3 23.3 Data
  - 1.26.4 23.4 Audit & Compliance
  - 1.26.5 23.5 Operations

1 System Design Document (SDD)

1.1 Guess X — Lowest Unique Bid Auction Platform

1.2 Cover Page

Field	Value
Document Title	System Design Document — Guess X Platform
Document Version	3.0
Document Date	2026-08-18
Document Status	Approved for Submission
Document Classification	Confidential — Proprietary
Prepared By	Guess X Engineering — Architecture Team
Document Type	Technical Architecture & Design Specification

1.2.1 Document Control

Version	Date	Author	Reviewer	Approver	Summary of Changes
1.0	2026-06-02	Architecture Team	Tech Lead	VP Engineering	Initial system design; monolith-style component inventory
2.0	2026-07-09	Architecture Team	Security Working	VP Engineering	Microservices decomposition,

Version	Date	Author	Reviewer	Approver	Summary of Changes
			Group		Redis HA, audit logging, DR runbook
3.0	2026-08-18	Architecture Team	Security & Platform Review Board	CTO	Enterprise hardening: Prisma ORM migration, Better-auth identity, ADRs, capacity planning, threat model, data governance, integration architecture, operational readiness

1.2.2 Distribution List

Recipient / Role	Organization	Purpose
CTO	Guess X	Executive accountability and sign-off
VP Engineering	Guess X	Delivery ownership
Security & Platform Review Board	Guess X	Security and operational review
Site Reliability Engineering (SRE)	Guess X	Operational readiness and on-call runbook ownership
Data Protection Officer (DPO)	Guess X	Data governance and privacy compliance
Architecture Review Board	Guess X	Architectural conformance

Recipient / Role	Organization	Purpose
		and ADR governance
Formal Submission Archive	Guess X	Retained as authoritative reference

1.2.3 Confidentiality Notice

This document is the proprietary and confidential property of Guess X. It contains sensitive architectural, security, and operational information. Unauthorized review, dissemination, distribution, copying, or use of the contents of this document, in whole or in part, is strictly prohibited. Access is limited to the named recipients on the distribution list and authorized reviewers. Retention is indefinite under the document control policy. If you have received this document in error, notify the originator immediately and destroy all copies.

1.3 Table of Contents

1. System Overview
2. High-Level Architecture
3. Component Diagrams and Interaction Flows
4. Deployment Architecture
5. Data Architecture
6. Scalability and Performance
7. Security Architecture
8. Technology Stack
9. Redis High Availability
10. Comprehensive Audit Logging
11. Backup and Disaster Recovery
12. WebSocket Horizontal Scaling
13. Architecture Decision Records (ADR)
14. Capacity Planning
15. Security Architecture Details
16. Operational Architecture
17. Data Governance
18. Integration Architecture
19. Feature Architecture Deep Dive
20. Performance Benchmarks

21. Production Deployment Topology
22. Disaster Recovery Runbook Summary
23. Security Hardening Checklist

1.4 1. System Overview

Guess X is a **microservices-based reverse auction platform** built on a **Lowest Unique Bid (LUB)** model. The architecture follows a **layered, service-oriented design** with clear separation of concerns across three backend services, two frontend applications, and a distributed data layer. The platform is engineered for enterprise-grade availability, security by design, and operational readiness under sustained high-concurrency bidding workloads.

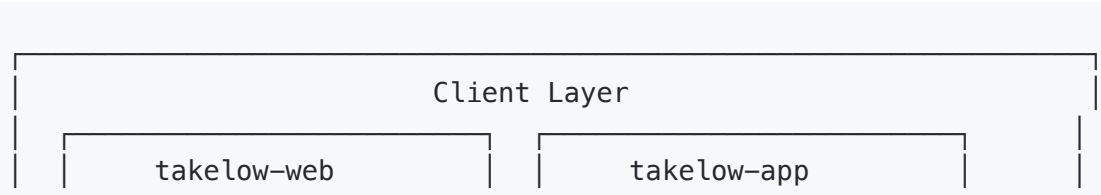
1.4.1 1.1 Design Principles

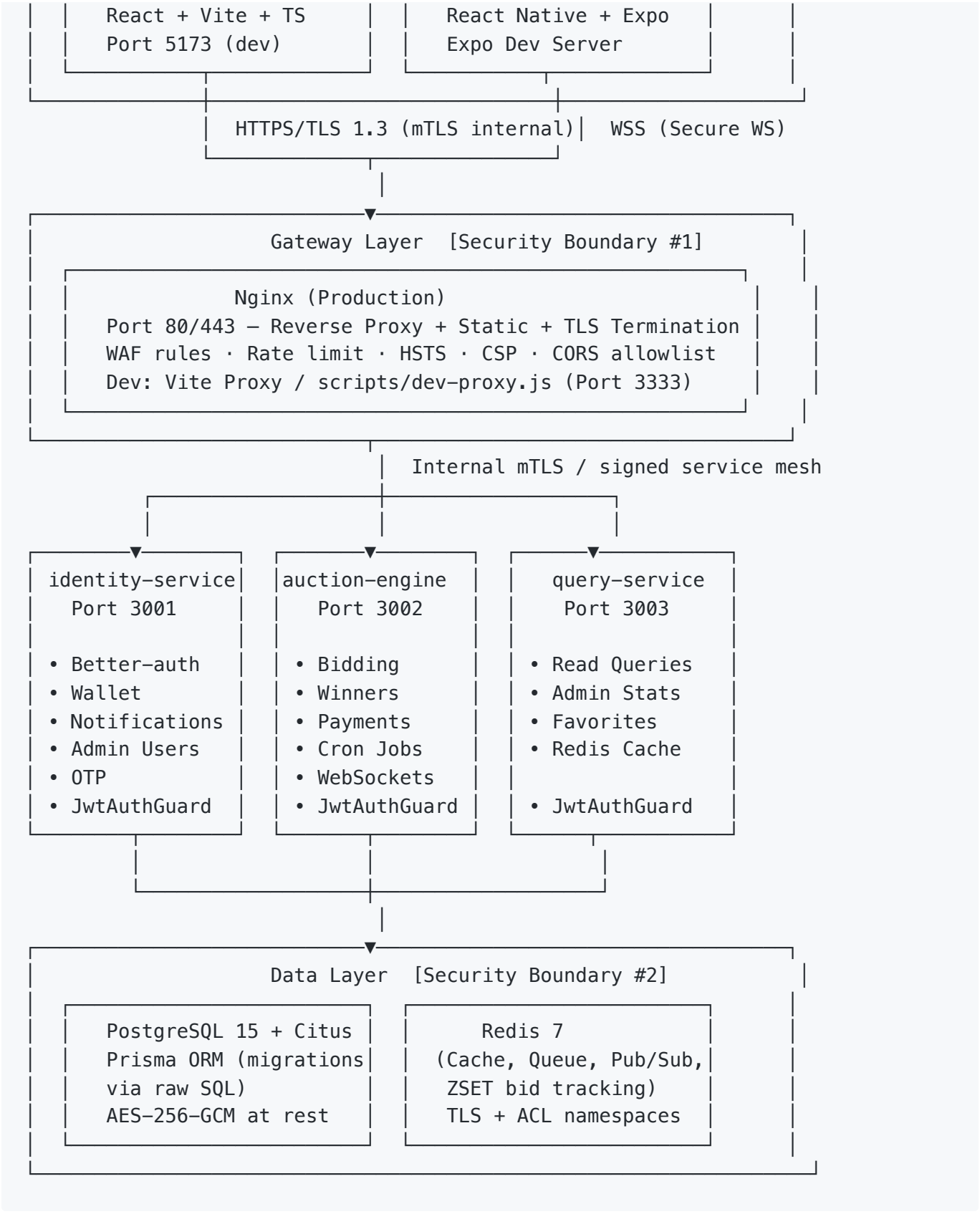
- **Separation of Concerns**: Each microservice owns a distinct domain (identity, bidding, query)
- **Read/Write Splitting**: Query Service handles all reads; Auction Engine handles writes and complex business logic
- **Stateless Services**: All backend services are stateless, enabling horizontal scaling
- **Event-Driven Real-Time**: Socket.io for live auction updates; BullMQ for async background processing
- **Citus-First Distribution**: Database designed for distributed PostgreSQL from day one
- **Security by Default**: Better-auth identity, JwtAuthGuard enforcement, rate limiting, nonce validation, HMAC webhooks, AES-256-GCM bid encryption, immutable audit logging
- **Defense in Depth**: Layered controls across network, identity, application, data, and audit boundaries
- **Operational Readiness**: Codified runbooks, SLO-driven alerting, automated failover, and tested disaster recovery

---

1.5 2. High-Level Architecture

1.5.1 2.1 Architecture Layers



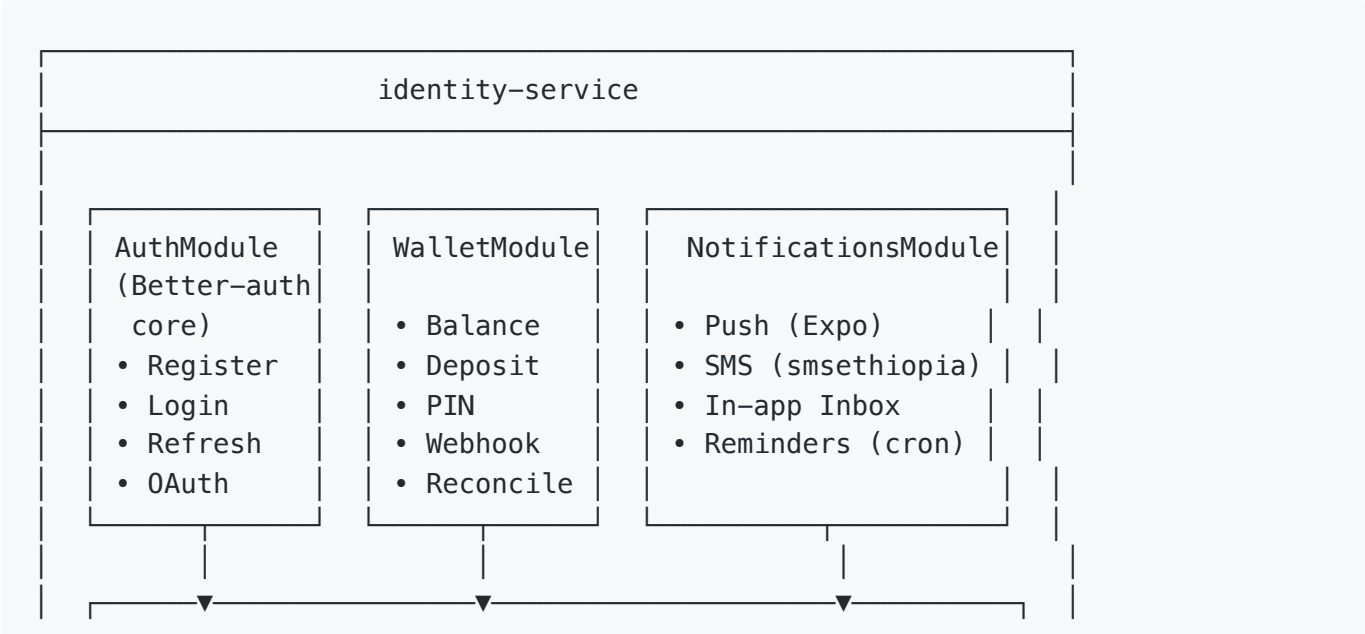


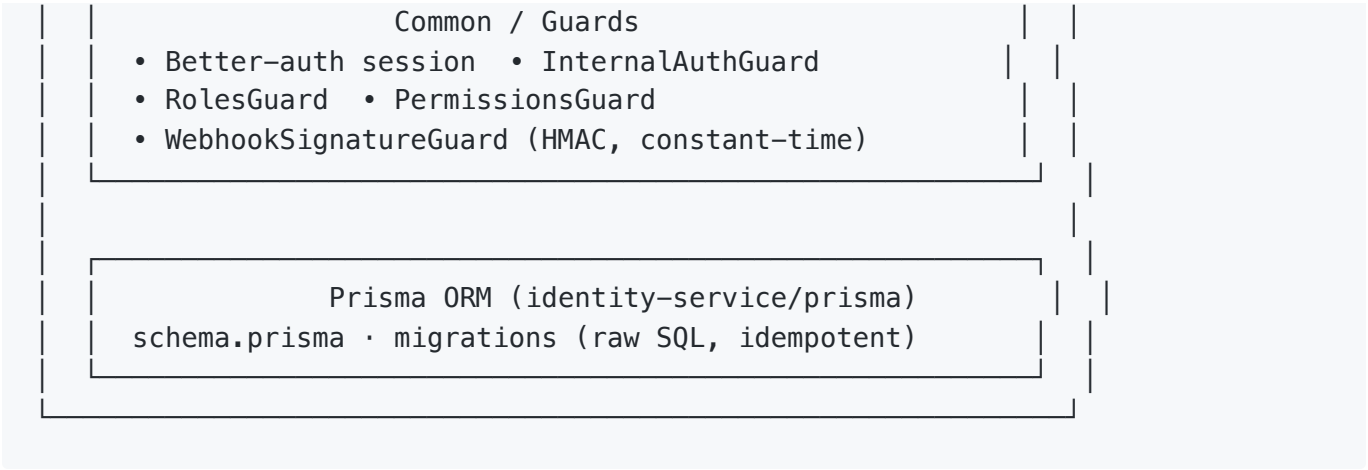
1.5.2 2.2 Layer Responsibilities

Layer	Responsibility	Technology
Client	UI rendering, state management, real-time updates	React, React Native, Tailwind, Socket.io Client
Gateway	Routing, CORS, static assets, SSL/TLS termination, WAF	Nginx, Vite dev proxy, http-proxy-middleware
Identity Service	Authentication, user management, wallet, notifications, admin users	NestJS, Prisma ORM, Better-auth, bcrypt
Auction Engine	Core bidding logic, winner calculation, payments, cron jobs, WebSocket	NestJS, Prisma ORM, BullMQ, Redis ZSETs, Socket.io, JwtAuthGuard, AES-256-GCM
Query Service	Read-optimized queries, caching, admin statistics	NestJS, Prisma ORM, CacheInterceptor, JwtAuthGuard, Redis caching layer
Data Layer	Persistent storage, distributed queries, caching, job queues	PostgreSQL + Citus, Prisma ORM, Redis

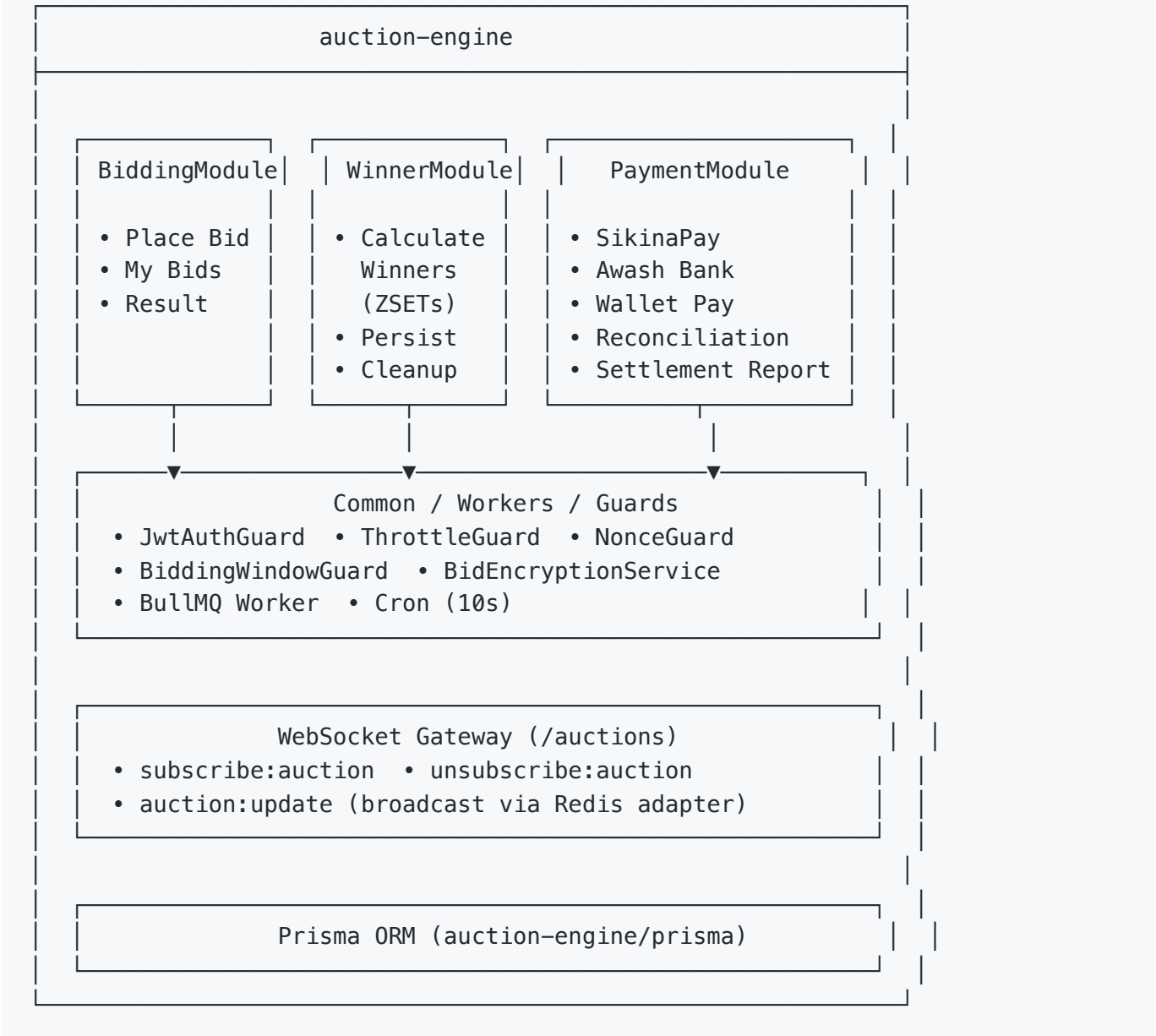
1.6 3. Component Diagrams and Interaction Flows

1.6.1 3.1 Identity Service Components

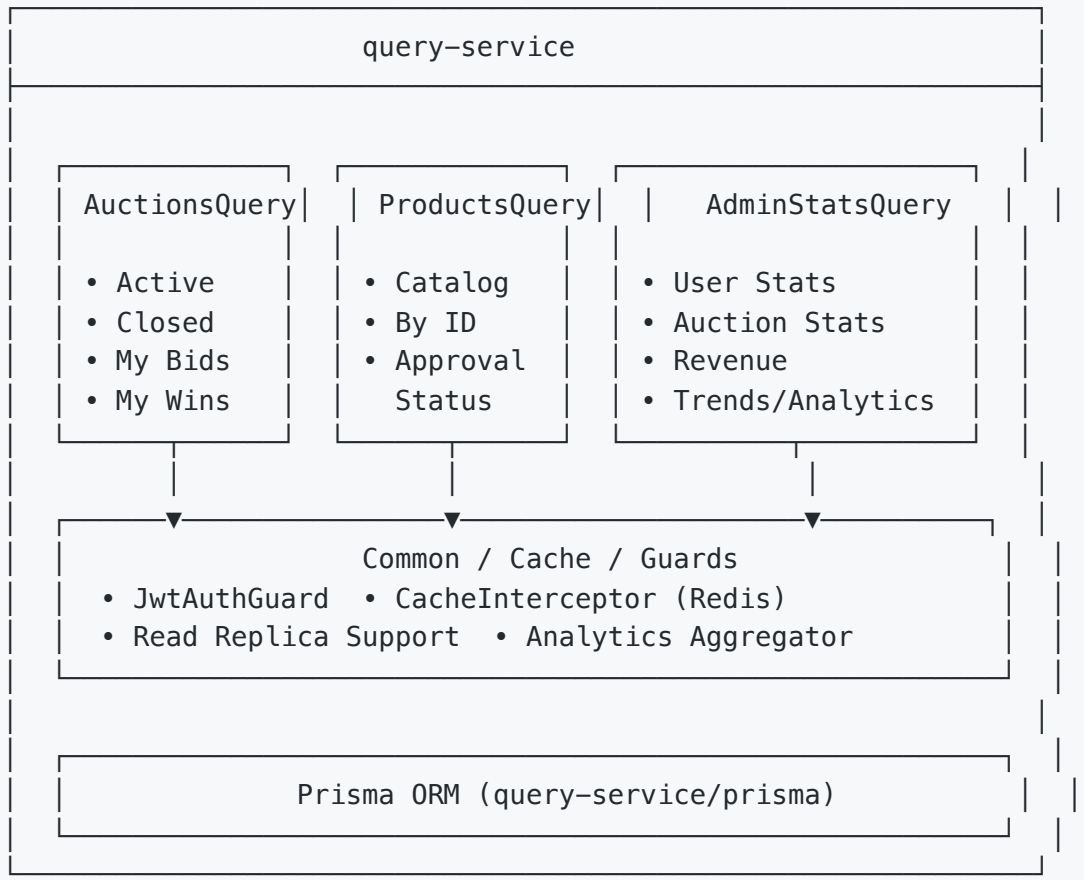




1.6.2 3.2 Auction Engine Components

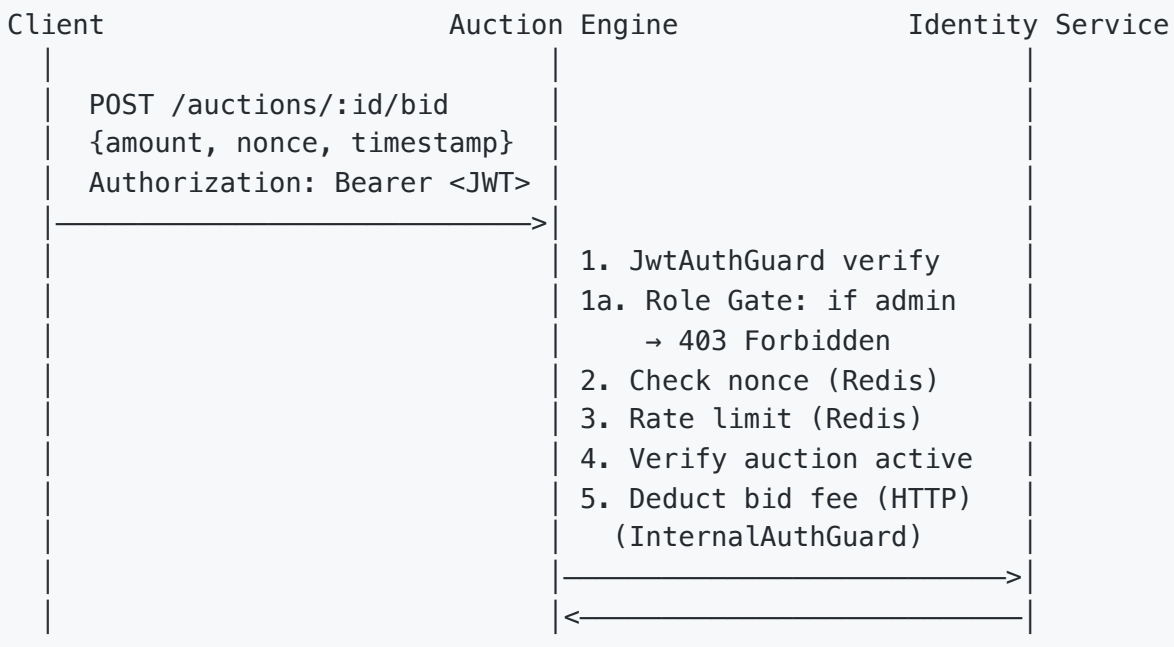


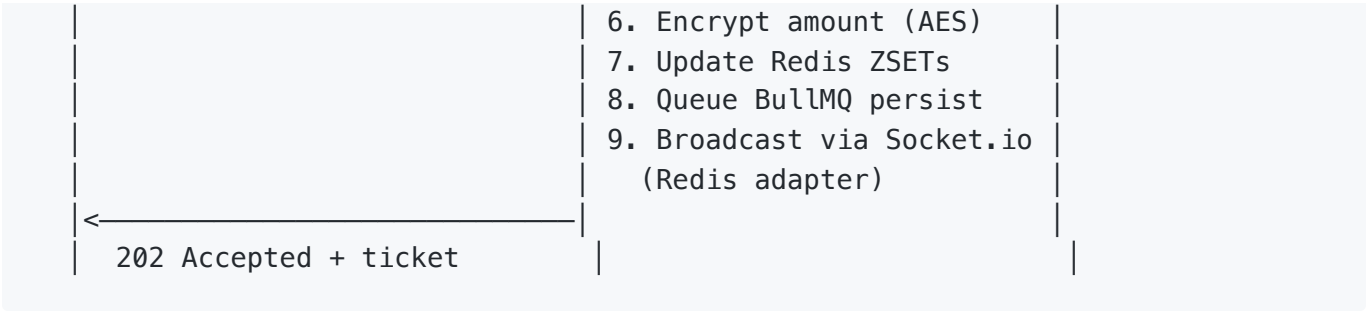
1.6.3 3.3 Query Service Components



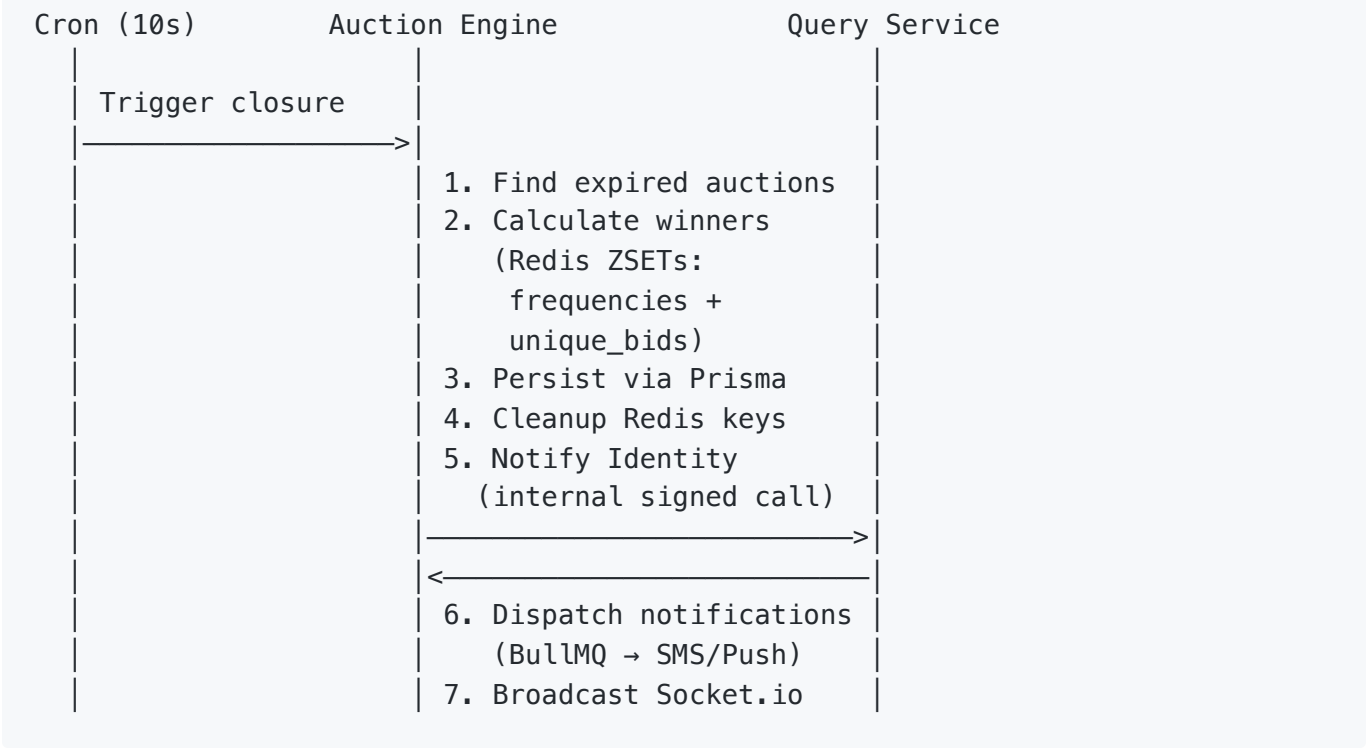
1.6.4 3.4 Key Interaction Flows

1.6.4.1 3.4.1 Bid Placement Flow

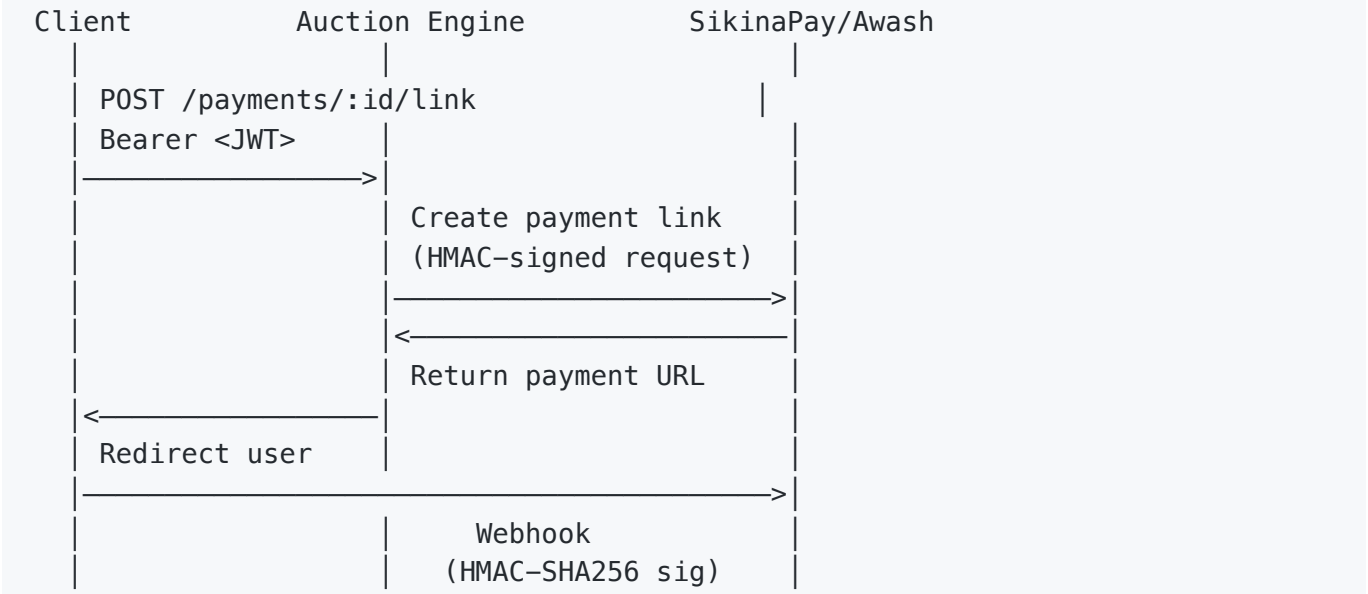




1.6.4.2 3.4.2 Winner Determination Flow



1.6.4.3 3.4.3 Payment Flow



```
   |                    |<──────────────────────────|
   |                    | Verify signature          |
   |                    |   (timingSafeEqual)       |
   |                    | Update status (Prisma)    |
   |                    | Notify winner             |
```

1.6.4.4 3.4.4 User Registration Flow

```
Client                      Identity Service              Database
   |                              |                          |
   |  POST /auth/register         |                          |
   |  {phone, name, password}     |                          |
   |─────────────────────────────>|                          |
   |                              | 1. Validate input        |
   |                              | 2. Check phone unique    |
   |                              |   (Prisma)               |
   |                              |─────────────────────────>|
   |                              |<─────────────────────────|
   |                              | 3. bcrypt hash password  |
   |                              | 4. Insert user (Prisma)  |
   |                              |─────────────────────────>|
   |                              |<─────────────────────────|
   |                              | 5. Better-auth session   |
   |                              | 6. Return tokens + user  |
   |<─────────────────────────────|                          |
```

1.6.4.5 3.4.5 Wallet Deposit Flow

```
Client          Identity Service          Payment Gateway
   |                    |                        |
   | POST /wallet/deposit                        |
   |──────────────────>|                         |
   |                    | Create pending txn     |
   |                    |   (Prisma)             |
   |                    |──────────────────────>|
   |                    |<──────────────────────|
   |                    | Return payment URL     |
   |<──────────────────|                         |
   | Redirect user      |                        |
   |──────────────────────────────────────────────>|
   |                    |     Webhook            |
   |                    |   (HMAC-SHA256 sig)    |
   |                    |<──────────────────────|
   |                    | Verify signature       |
   |                    |   (timingSafeEqual)    |
```

```
   |                    | Update balance (Prisma) |
   |                    | Confirm transaction     |
```

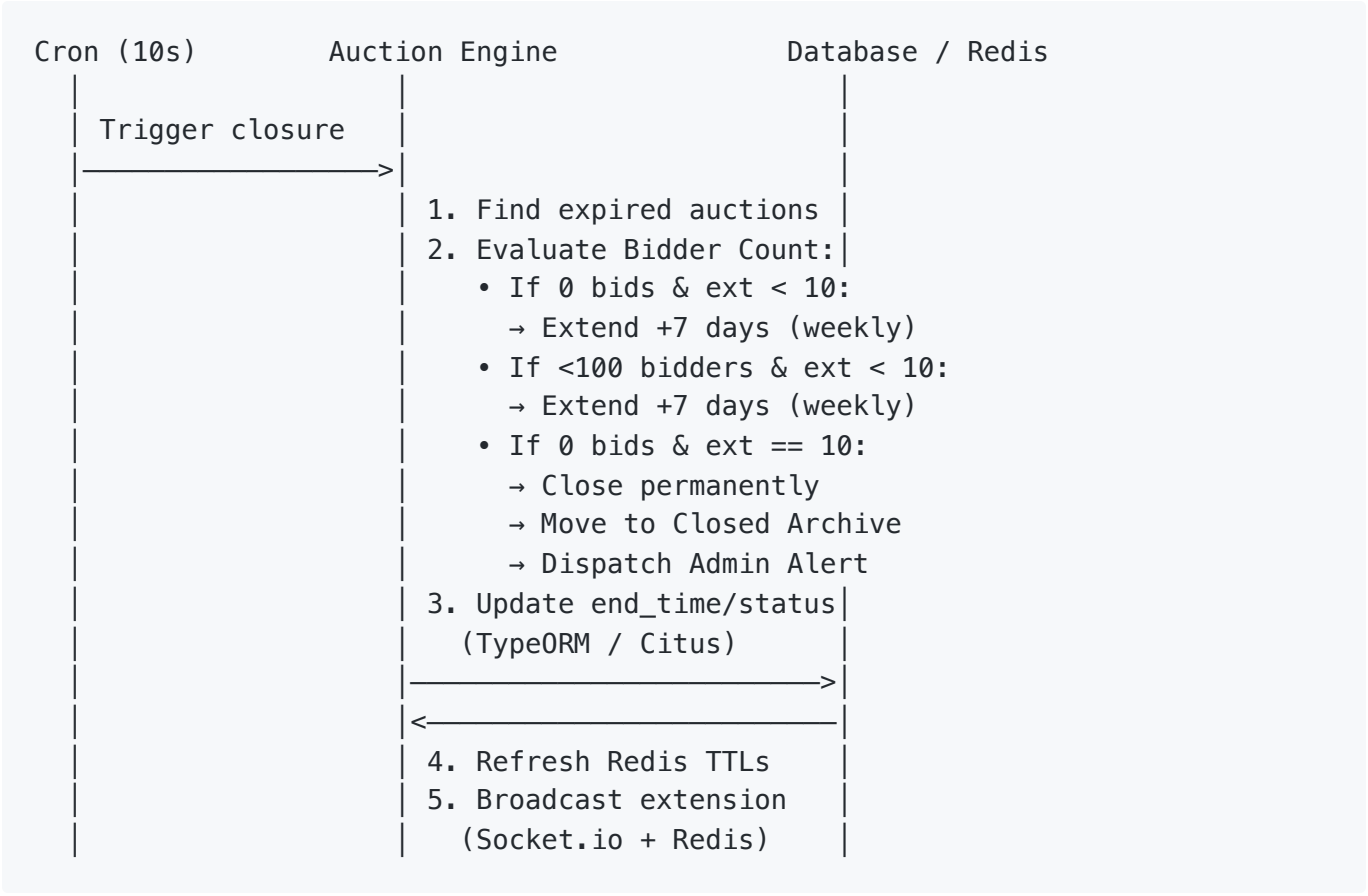
1.6.4.6 3.4.6 Admin Auction Creation Flow

```
Admin              Auction Engine           Query Service
   |                     |                        |
   | POST /admin/auctions                          |
   | Bearer <JWT>        |                        |
   |-------------------->|                        |
   |                     | 1. JwtAuthGuard +      |
   |                     |    RolesGuard (admin)  |
   |                     | 2. Validate product    |
   |                     | 3. Create auction      |
   |                     |   (Prisma)             |
   |                     | 4. Invalidate caches   |
   |                     |----------------------->|
   |                     |<-----------------------|
   |                     | 5. Return auction      |
   |<--------------------|                        |
```

1.6.4.7 3.4.7 Notification Dispatch Flow

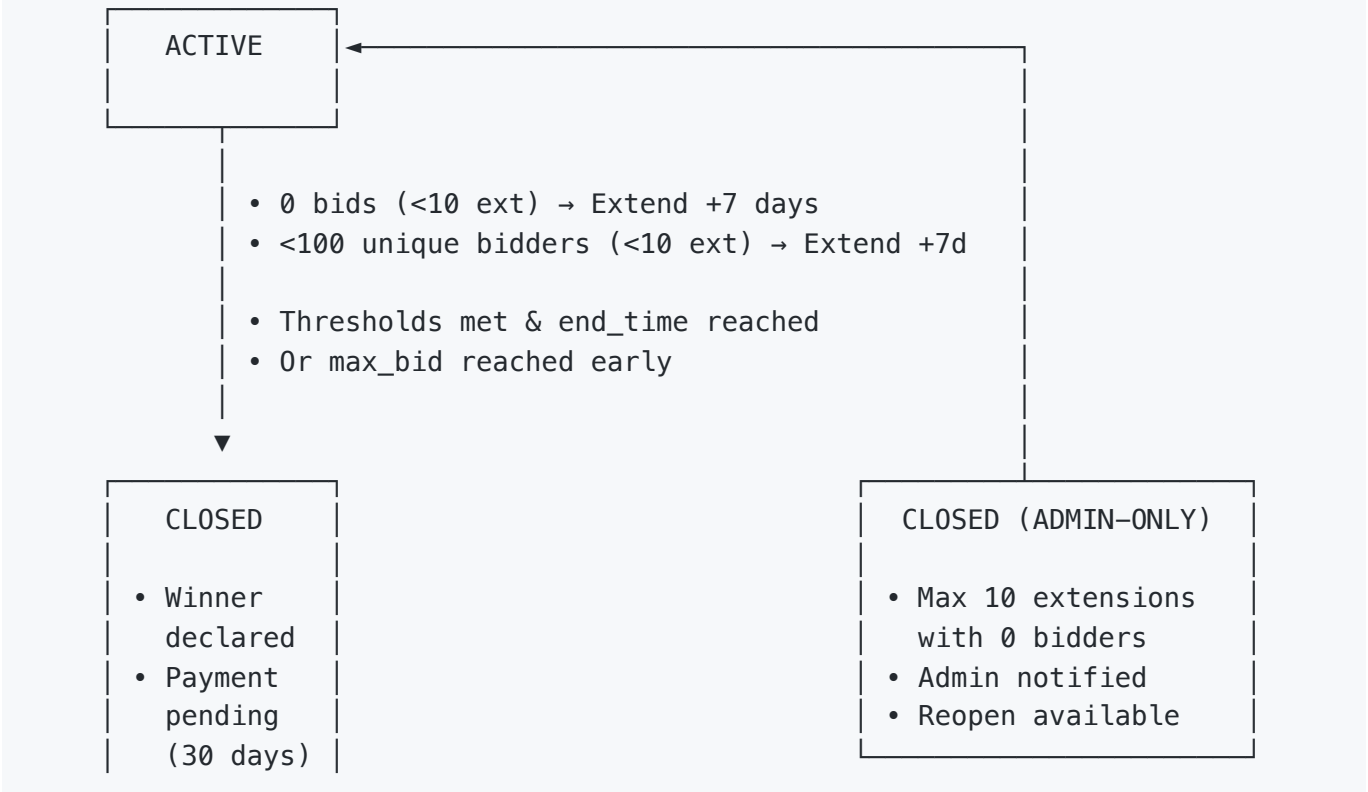
```
Auction Engine     Identity Service         Redis/External
   |                     |                        |
   | POST /notify/winner (internal)               |
   | (InternalAuthGuard + HMAC)                   |
   |-------------------->|                        |
   |                     | 1. Queue notification  |
   |                     |    (BullMQ)            |
   |                     | 2. Save to DB (Prisma) |
   |                     |----------------------->|
   |                     |<-----------------------|
   |                     | 3. Return 202          |
   |<--------------------|                        |
   |                     |                        |
   |                     | BullMQ Worker:         |
   |                     | - Push via Expo        |
   |                     | - SMS via smsethiopia  |
   |                     | - In-app inbox         |
```

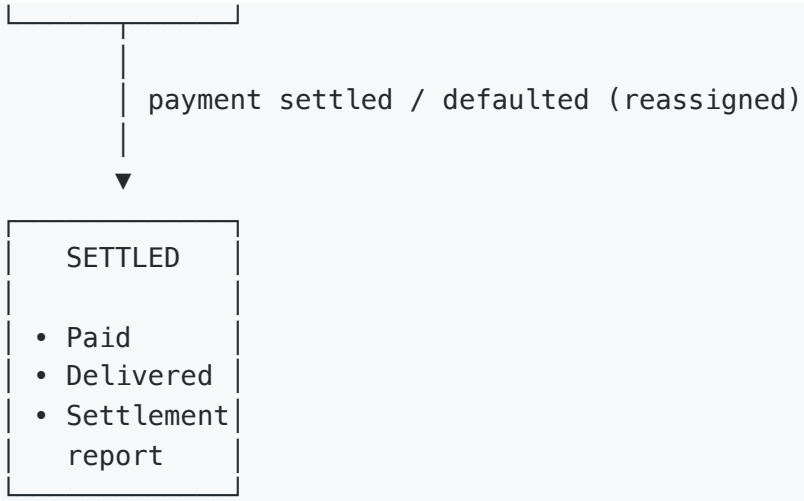
1.6.4.8 3.4.8 Auction Extension & Closure Trigger Flow



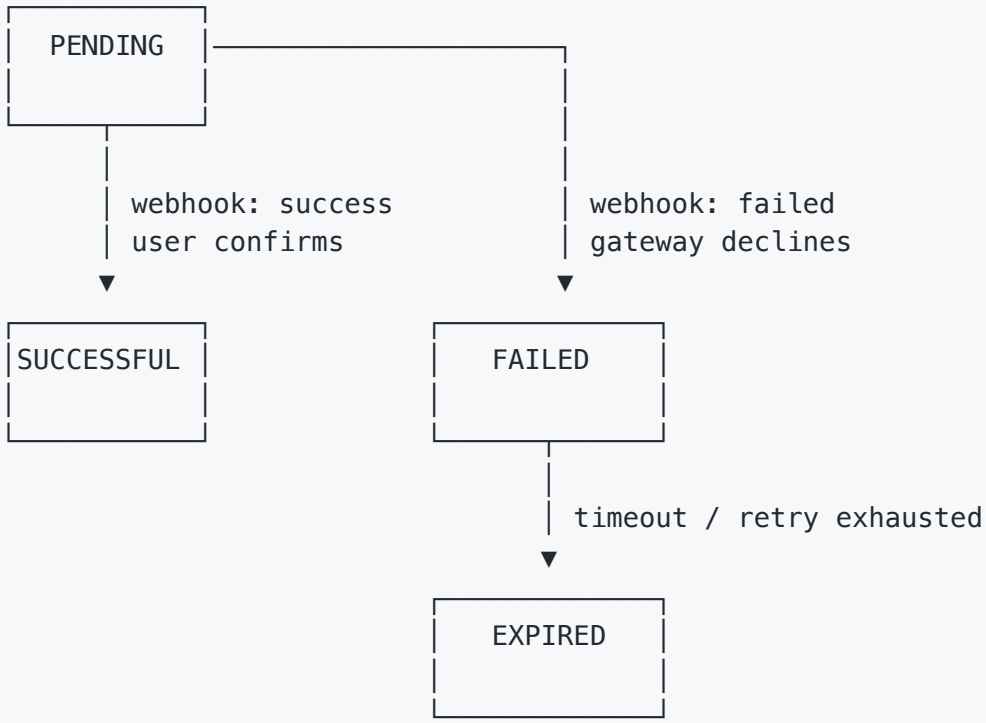
1.6.5 3.5 State Diagrams

1.6.5.1 3.5.1 Auction State Machine

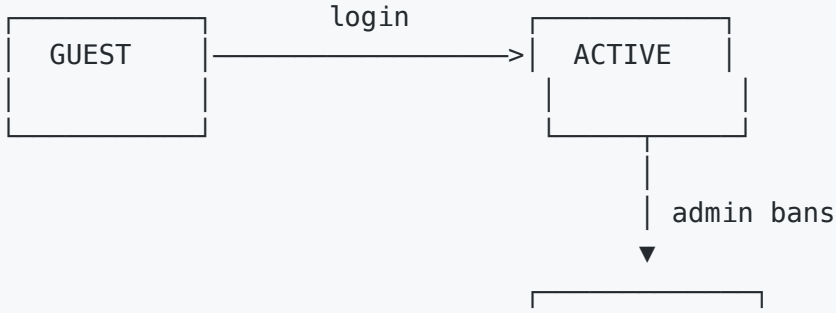




1.6.5.2 3.5.2 Payment Transaction State Machine



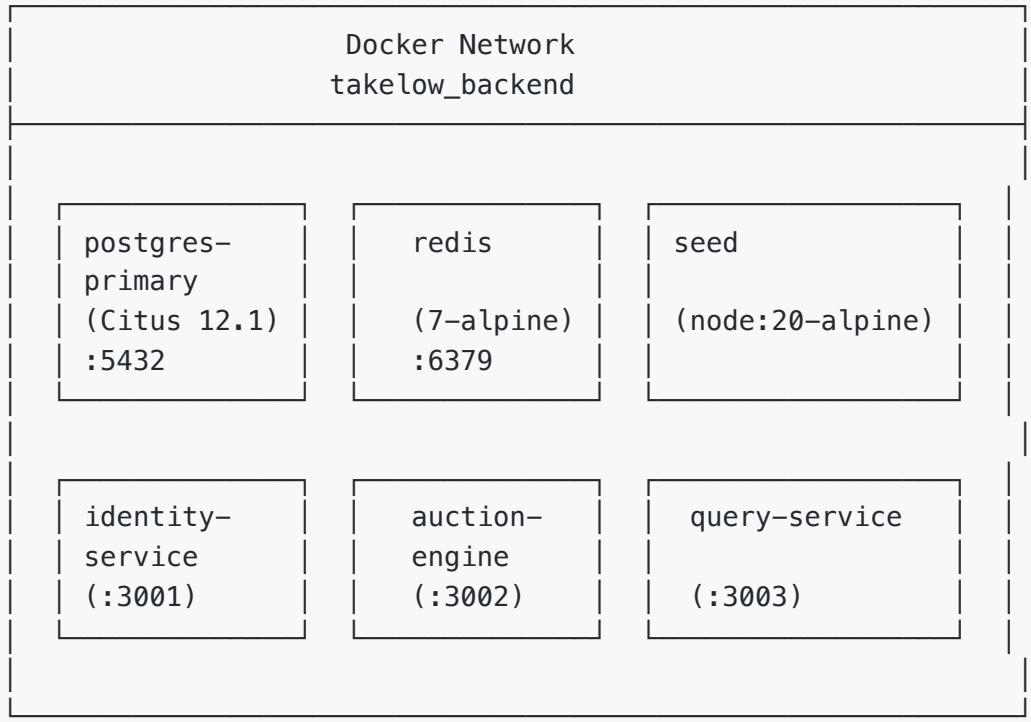
1.6.5.3 3.5.3 User Account State Machine



```
                                    │   BANNED    │
                                    │             │
                                    └─────────────┘
```

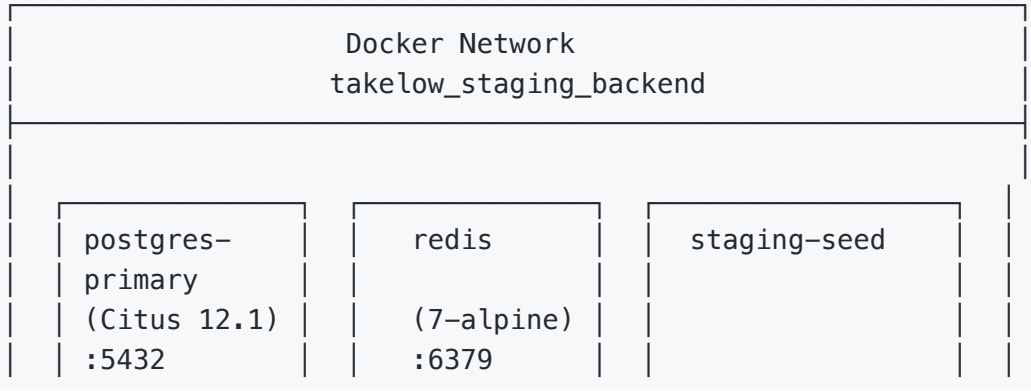
1.7 4. Deployment Architecture

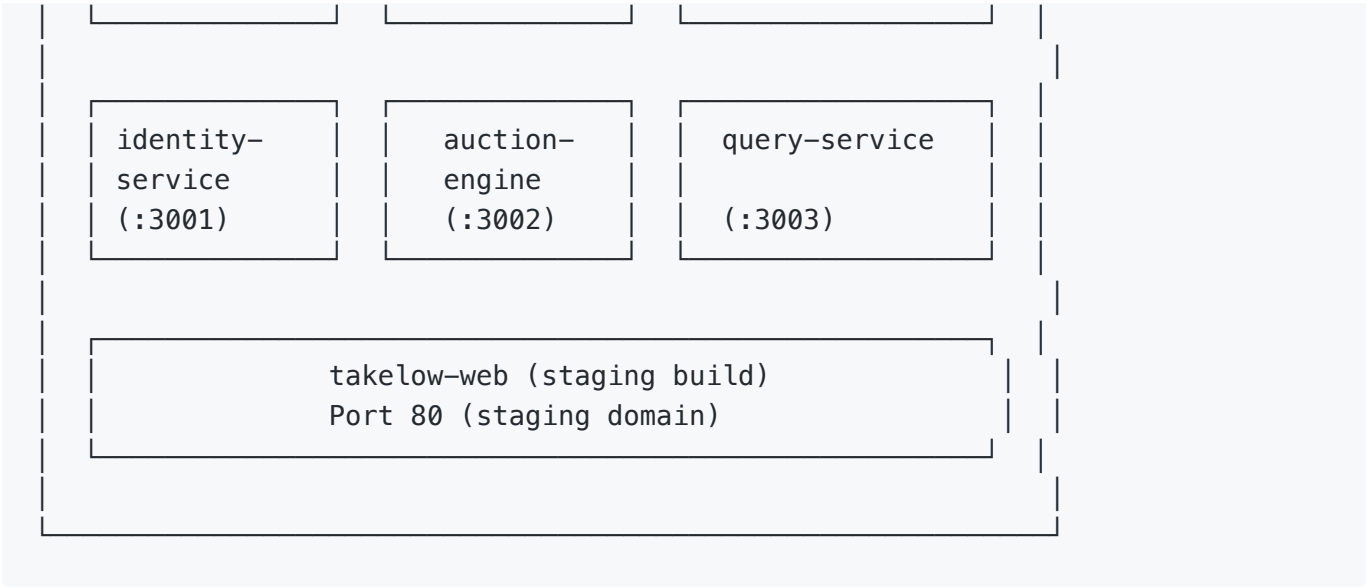
1.7.1 4.1 Development Environment (Docker Compose)



Docker Compose Networks: - `takelow_backend` : All backend services + database (bridge driver) - `takelow_frontend` : Web frontend + backend services (bridge driver)

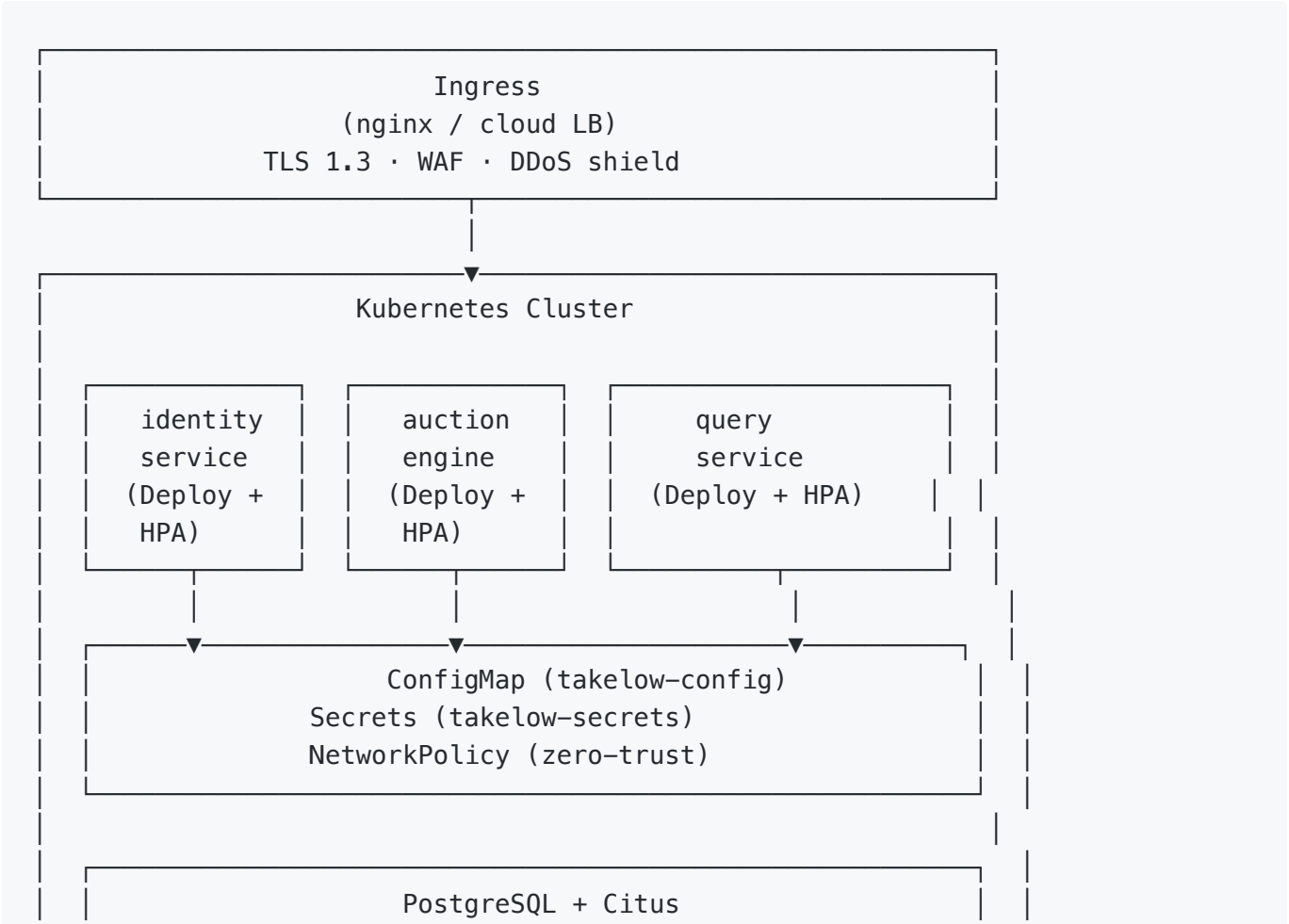
1.7.2 4.2 Staging Environment





Staging Characteristics: - Mirrors production Kubernetes configuration but runs on Docker Compose for QA/UAT - Uses production-like data volumes (seeded from anonymized production snapshots) - Exposed via staging domain with TLS termination - Separate secrets namespace from production - Automated deployment via CI/CD staging pipeline - Used for: integration testing, UAT, performance benchmarking, security scanning

1.7.3 4.3 Production Environment (Kubernetes)



```
│   │                  (Managed Service / StatefulSet)        │   │
│   │                  Daily backups, PITR enabled            │   │
│   │                  AES-256-GCM at rest, TLS in transit    │   │
│   └─────────────────────────────────────────────────────────┘   │
│                                                                 │
│   ┌─────────────────────────────────────────────────────────┐   │
│   │                        Redis Cluster                    │   │
│   │                  (Managed Service / StatefulSet)        │   │
│   │                  Persistence + Replication + TLS        │   │
│   └─────────────────────────────────────────────────────────┘   │
└─────────────────────────────────────────────────────────────────┘
```

1.7.4 4.4 Kubernetes Manifests

Manifest	Purpose
`k8s/config.yaml`	ConfigMap for environment variables
`k8s/deployments.yaml`	Deployments for all 3 services + web
`k8s/hpa.yaml`	Horizontal Pod Autoscaling (min 2, max 10)
`k8s/ingress.yaml`	Ingress rules with TLS
`k8s/migrate-job.yaml`	Kubernetes Job for database migrations
`k8s/web.yaml`	Nginx deployment for frontend
`k8s/networkpolicy.yaml`	Zero-trust pod-to-pod traffic policy
`k8s/pdb.yaml`	Pod Disruption Budgets for voluntary evictions
`k8s/backup-cronjob.yaml`	Scheduled backup CronJobs

1.7.5 4.5 Disaster Recovery and Backup

1.7.5.1 4.5.1 Backup Strategy

Component	Backup Method	Frequency	Retention	Storage
PostgreSQL	pg_dump / pg_basebackup	Daily full + WAL archiving	30 days	Object storage (S3/equivalent)
Redis	RDB snapshots	Every 6 hours	7 days	Persistent volume +

Component	Backup Method	Frequency	Retention	Storage
				object storage
Kubernetes	etcd snapshots	Daily	30 days	Object storage
Secrets	Kubernetes Secrets manifests	On change	Git history	Private Git repository

1.7.5.2 4.5.2 Recovery Objectives

Metric	Target	Strategy
RTO (Recovery Time Objective)	< 4 hours	Automated Kubernetes deployment from Git; database restore from latest backup
RPO (Recovery Point Objective)	< 24 hours	Daily database backups + continuous WAL archiving
RTO (Data corruption)	< 1 hour	Point-in-time recovery (PITR) using WAL archives

1.7.5.3 4.5.3 Failover Procedures

1. **Database Failover**:
    - Promote read replica to primary (if using managed service with automatic failover)
    - Update `DATABASE_URL` in ConfigMap
    - Rolling restart of all services
2. **Redis Failover**:
    - Redis Sentinel or Cluster automatic failover
    - Services reconnect via DNS
3. **Application Failover**:
    - Kubernetes HPA ensures minimum replicas
    - Pod disruption budgets prevent simultaneous evictions
    - Rolling updates with health check gates

1.7.6 4.6 Monitoring and Alerting

1.7.6.1 4.6.1 Observability Stack

Component	Technology	Purpose
Metrics	Prometheus + Grafana	Service metrics, DB/Redis stats, custom business metrics
Logging	ELK Stack (Elasticsearch, Logstash, Kibana)	Centralized structured JSON logs
Tracing	OpenTelemetry + Jaeger	Distributed tracing across microservices
Alerting	Alertmanager	PagerDuty, Slack, email notifications

1.7.6.2 4.6.2 Key Metrics and Alerts

Metric	Alert Threshold	Severity
HTTP error rate (5xx)	> 1% for 5 minutes	Critical
HTTP latency p99	> 500ms for 5 minutes	Warning
Database connections	> 80% of pool for 2 minutes	Warning
Redis memory usage	> 85% for 5 minutes	Warning
BullMQ queue depth	> 1000 jobs for 10 minutes	Warning
Disk usage	> 85% for 5 minutes	Critical
Pod restart count	> 3 in 10 minutes	Critical
Health check failures	2 consecutive failures	Critical

1.7.6.3 4.6.3 SLI/SLO Definitions

SLI	SLO	SLA
API availability	99.9% monthly	99.9%
API latency (p95)	< 200ms	< 500ms
WebSocket connection success	99.5%	99.0%
Payment webhook processing	< 30s	< 60s

SLI	SLO	SLA
Database query p95	< 100ms	< 250ms

1.7.7 4.7 CI/CD Pipeline

```
Code Commit
    │
    ▼
GitHub Actions / GitLab CI
    │
    ├──> Lint (ESLint, Prettier)
    ├──> Type Check (tsc --noEmit)
    ├──> Unit Tests (Jest)
    ├──> Security Scan (Snyk, npm audit, Trivy image scan)
    ├──> SAST/DAST Scan (Semgrep, OWASP ZAP)
    ├──> Build Docker Images
    │       ├── identity-service
    │       ├── auction-engine
    │       ├── query-service
    │       └── guessx-web
    │
    ▼
Push to Registry (ghcr.io)
    │
    ▼
Deploy to Staging
    │
    ├──> Integration Tests
    ├──> E2E Tests (Cypress/Playwright)
    ├──> Load Tests (k6)
    │
    ▼ (manual approval gate)
Deploy to Production
    │
    ├──> Migrate Job (schema_migrations, idempotent)
    ├──> Deploy identity-service (canary)
    ├──> Deploy auction-engine (canary)
    ├──> Deploy query-service (canary)
    └──> Deploy web (nginx)
```

1.8 5. Data Architecture

1.8.1 5.1 Database Strategy

Aspect	Decision
Primary DB	PostgreSQL 15 with Citus 12.1 extension
ORM	Prisma ORM with raw SQL migrations (idempotent, `IF NOT EXISTS`)
Migration Strategy	Raw SQL files tracked via `schema_migrations` table; applied via `scripts/migrate-raw.sh`
Schema Source of Truth	`prisma/schema.prisma` per service + raw SQL migrations for Citus distribution
Distribution	Citus distributed table on `bids.user_id`
Partitioning	Native PostgreSQL range partitioning on `bids_partitioned.bid_time`
Read Scaling	`READ_REPLICA_URL` support in query-service
Caching	Redis for sessions, rate limits, bid tracking, BullMQ queues, query cache

1.8.2 5.2 Citus Distribution Strategy

```
Coordinator Node (takelow_postgres_primary)
    │
    ├──> Worker Node 1 (users, products, auctions, winners)
    ├──> Worker Node 2 (bids distributed by user_id)
    └──> Worker Node 3 (payment_transactions, notification_logs)
```

Reference Tables (replicated to all workers): - `users` - `products` - `auctions` - `otps`

Distributed Tables (sharded by hash): - `bids` — distributed by `user_id`

1.8.3 5.3 Redis Usage

Key Pattern	Purpose	TTL
`login:ratelimit:{identifier}`	Login rate limiting	60s
`bid:nonce:{nonce}`	Bid replay prevention	60s
`bid:throttle:{userId}`	Bid rate limiting	1s
`guessx:auction:{id}:lock`	Auction write lock	30s

Key Pattern	Purpose	TTL
`guessx:auction:{id}:bids`	Active bid count	Auction duration
`guessx:banned-users`	Banned user blocklist	7 days
`frequencies:{auctionId}`	ZSET — bid amount frequencies	Auction duration
`unique_bids:{auctionId}`	ZSET — unique bid amounts	Auction duration
`socket.{userId}`	Socket.io session mapping	12h
`cache:query:{hash}`	Query-service response cache	60–300s
`terms:accepted:{userId}:{version}`	T&C acceptance flag	Until version change

1.9 6. Scalability and Performance

1.9.1 6.1 Horizontal Scaling

Component	Scaling Strategy
Identity Service	Kubernetes HPA based on CPU/memory; stateless
Auction Engine	Kubernetes HPA; Redis adapter for Socket.io scaling
Query Service	Kubernetes HPA + PostgreSQL read replicas + Redis cache
PostgreSQL	Citus worker node addition; rebalance distributed tables
Redis	Redis Cluster mode for sharding

1.9.2 6.2 Performance Optimizations

Optimization	Implementation
Database Indexing	20+ indexes including partial, composite, and unique constraints
Query Optimization	Prisma typed queries with batch aggregates; avoids N+1 queries
Caching	`CacheInterceptor` on query-service read endpoints (Redis)
Batch Persistence	BullMQ batch size 50 for bid writes
Connection Pooling	Prisma connection pool (configurable)
Compression	Gzip/Brotli via Nginx for static assets
CDN	Static assets served via CDN in production

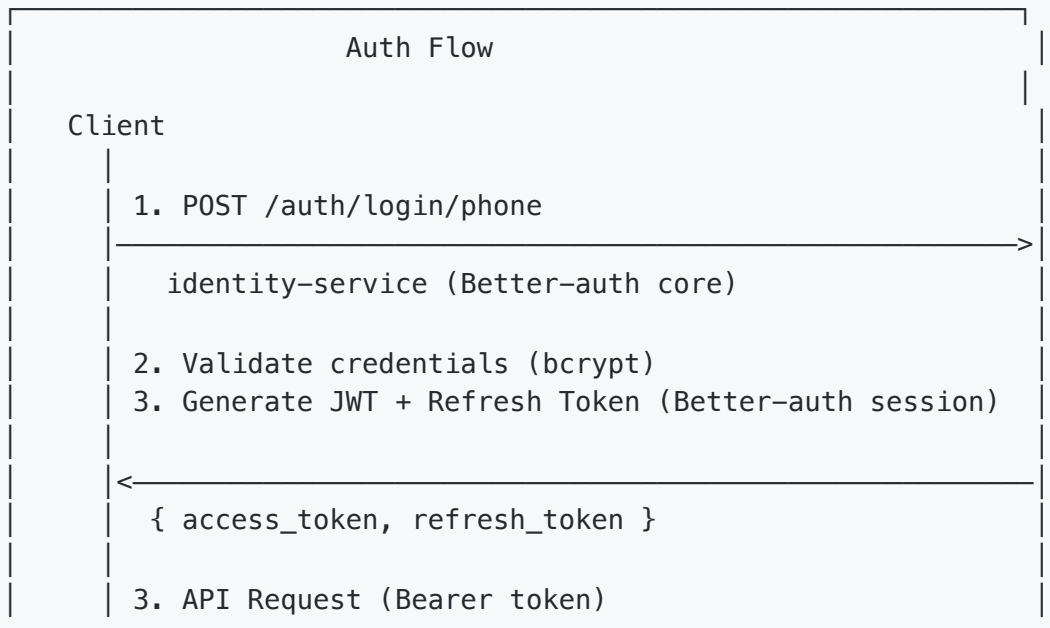
1.9.3 6.3 Load Testing

Load tests are located in `load-tests/` using k6: - `bid-flood.js`: Simulates high-concurrency bid submission - Scenarios: 1000 concurrent users, 100 bids/second target

---

1.10 7. Security Architecture

1.10.1 7.1 Authentication and Authorization



```
│      │──────────────────────────────────────────────────────────>│
│      │   auction-engine / query-service                         │
│      │                                                          │
│      │ 4. JwtAuthGuard validate JWT signature                   │
│      │    (shared JWT secret across all services)               │
│      │ 5. Check role/permissions (RolesGuard)                   │
│      │                                                          │
│      │<─────────────────────────────────────────────────────────│
│      │       Response                                           │
└──────────────────────────────────────────────────────────────────┘
```

Identity boundary: Tokens are issued by `identity-service` (Better-auth) and verified by all three services via `JwtAuthGuard`. The JWT secret must be identical across services.

Fail-Closed Policy: All microservices enforce a strict fail-closed authentication policy. If `JwtService` is missing, unconfigured, or fails token verification, `JwtAuthGuard` explicitly denies access with `401 Unauthorized` (`Authentication service unavailable`) rather than allowing unauthenticated passthrough.

1.10.2 7.2 Threat Mitigations

Threat	Mitigation
SQL Injection	Parameterized queries via Prisma; raw SQL with bindings
Auth Bypass / Misconfig	Fail-closed `JwtAuthGuard` strictly rejects requests when auth service is uninitialized
Double-Credit / Replay Deposit	Idempotent deposit handling with optional client `reference_id` and unique user-scoped transaction reference fallback
Duplicate Webhooks / Payment Race	State-safe idempotent payment confirmation (`markAsPaid`) gracefully handling re-deliveries on settled auctions
XSS	React auto-escaping; Content-Security-Policy headers
CSRF	HMAC-SHA256 CSRF tokens; SameSite cookies
SSRF	Image download validation (HTTPS only, private IP blocked)
Brute Force	Redis rate limiting (5 login attempts/min)

Threat	Mitigation
Replay Attacks	Nonce validation (Redis, 60s TTL); timestamp tolerance
Man-in-the-Middle	TLS 1.3 in production; HMAC webhook signatures
Data Breach	Encryption at rest (AES-256-GCM for bids); hashed credentials
Token Forgery	JWT signature verification (HS256); shared secret rotation
Supply Chain	Dependency scanning (Snyk, npm audit); pinned versions

1.10.3 7.3 Security Headers (Production)

- `Strict-Transport-Security`
- `X-Content-Type-Options: nosniff`
- `X-Frame-Options: DENY`
- `Content-Security-Policy`
- CORS with configurable allowed origins

1.11 8. Technology Stack

1.11.1 8.1 Backend Services

Component	Technology	Version
Runtime	Node.js	20.x
Framework	NestJS	10.x
Language	TypeScript	5.x
ORM	Prisma ORM	5.x
Database	PostgreSQL + Citus	15 / 12.1
Cache	Redis	7.x
Queue	BullMQ	5.x

Component	Technology	Version
Real-time	Socket.io	4.x
Auth (identity-service)	Better-auth	—
Auth (auction-engine, query-service)	JwtAuthGuard	—
Validation	class-validator	—
Cryptography	crypto (Node built-in)	—

1.11.2 8.2 Frontend Applications

Component	Technology	Version
Framework (Web)	React	19.x
Bundler	Vite	5.x
Language	TypeScript	5.x
Styling	Tailwind CSS	3.x
Animations	Framer Motion	12.x
Icons	Lucide React	—
State (Web)	React Context + useReducer	—
State (Mobile)	Zustand	5.x
Framework (Mobile)	React Native + Expo	0.86 / SDK 52
Real-time	Socket.io Client	4.x

1.11.3 8.3 Infrastructure

Component	Technology
Containerization	Docker, Docker Compose
Orchestration	Kubernetes
Reverse Proxy	Nginx
CI/CD	GitHub Actions / GitLab CI

Component	Technology
Registry	GitHub Container Registry (ghcr.io)
Monitoring	Prometheus + Grafana + OpenTelemetry
Logging	ELK Stack
Load Testing	k6
Secret Management	Kubernetes Secrets + sealed-secrets

1.11.4 8.4 External Services

Service	Purpose
SikinaPay	Payment gateway (sandbox + production)
Awash Bank	Payment gateway (sandbox + production)
SMS Ethiopia	SMS notifications
Expo Push	Mobile push notifications
TeleBirr	OAuth social login
Banking API	OAuth social login

1.11.5 8.5 Data Layer

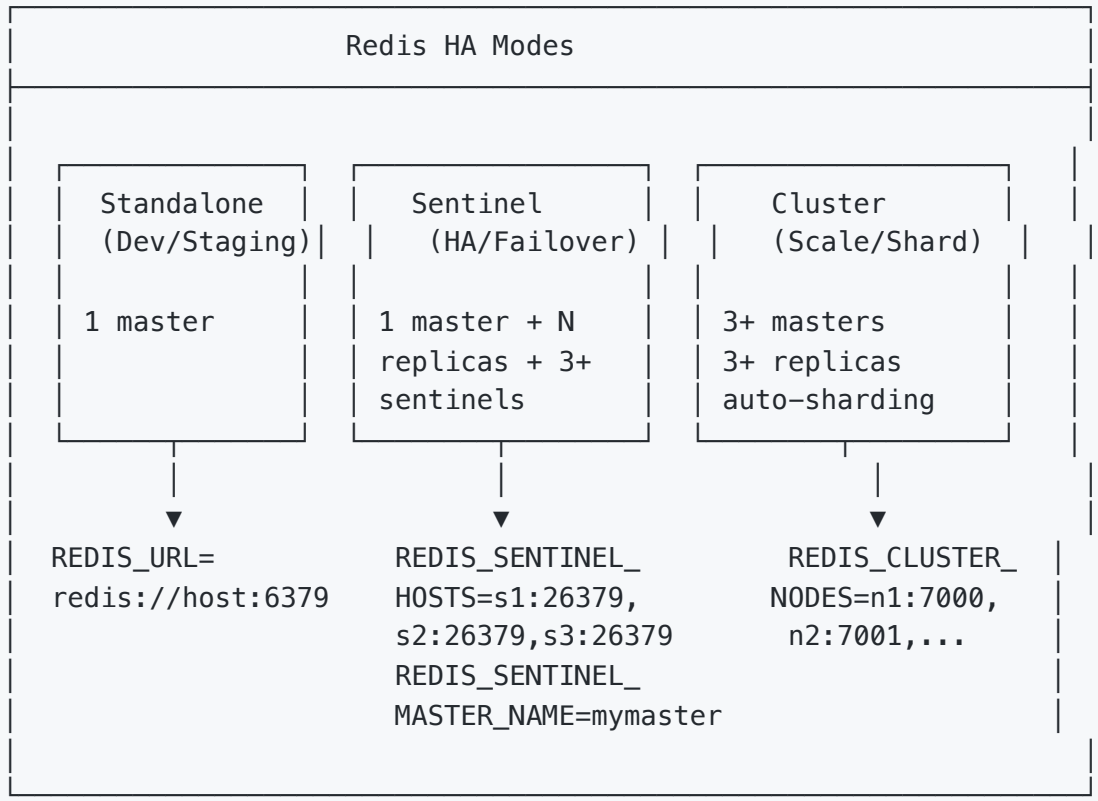
Layer	Technology	Role
Relational Store	PostgreSQL 15 + Citus 12.1	Persistent source of truth, distributed queries
ORM / Access	Prisma ORM	Typed data access, schema source of truth, migration generation
Cache & Coordination	Redis 7	Query cache, rate limits, nonce store, ZSET bid tracking, BullMQ queues, Socket.io Pub/Sub

Layer	Technology	Role
Encryption	AES-256-GCM (Node crypto)	Bid amount encryption at rest
Migrations	Raw SQL via `scripts/migrate-raw.sh`	Idempotent schema changes tracked in `schema_migrations`

1.12 9. Redis High Availability

1.12.1 9.1 Architecture Overview

The system supports three Redis deployment modes with automatic detection via environment variables:



1.12.2 9.2 Configuration Detection Logic

```
// Shared logic in each service's Redis provider
function parseRedisConfig(): RedisOptions {
```

```
  const sentinelHosts = process.env.REDIS_SENTINEL_HOSTS?.split(',');
  const sentinelMasterName = process.env.REDIS_SENTINEL_MASTER_NAME;
  const clusterNodes = process.env.REDIS_CLUSTER_NODES?.split(',');

  if (sentinelHosts && sentinelMasterName) {
    return { sentinel: { hosts: sentinelHosts, masterName: sentinelMasterName
        }};
  }
  if (clusterNodes) {
    return { cluster: { nodes: clusterNodes.map(n => parseNode(n)) }};
  }
  return { url: process.env.REDIS_URL || 'redis://localhost:6379' };
}
```

1.12.3 9.3 Sentinel Mode (Production Recommended)

Deployment (Docker Compose): - 1 Redis master + 2 replicas - 3 Sentinel instances for quorum - Master announced via `replica-announce-ip` / `replica-announce-port`

Failover Behavior: - Sentinel quorum: 2/3 sentinels must agree - Failover timeout: 60 seconds - Client reconnection: Automatic via ioredis Sentinel support

Environment Variables:

```
REDIS_SENTINEL_HOSTS=sentinel-1:26379,sentinel-2:26379,sentinel-3:26379
REDIS_SENTINEL_MASTER_NAME=guessx-master
REDIS_SENTINEL_ROLE=master
```

1.12.4 9.4 Cluster Mode (High Scale)

Deployment: - 6 nodes minimum (3 masters + 3 replicas) - Hash slots: 16384 distributed across masters - Automatic resharding via `redis-cli --cluster reshard`

Environment Variables:

```
REDIS_CLUSTER_NODES=node-1:7000,node-2:7001,node-3:7002,node-4:7003,node-5:7004,n
```

Scaling: - Add nodes: `redis-cli --cluster add-node` - Rebalance: `redis-cli --cluster reshard` - Read scaling: `scaleReads: 'slave'` in ioredis Cluster options

1.12.5 9.5 Kubernetes Deployment

See `docs/REDIS_HA.md` for complete Kubernetes manifests including: - StatefulSet for Redis master/replica - Sentinel deployment with ConfigMap - Cluster mode with 6-node StatefulSet - Health checks and Prometheus metrics

---

1.13 10. Comprehensive Audit Logging

1.13.1 10.1 Audit Log Entity

```
CREATE TABLE audit_logs (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    actor_id VARCHAR NOT NULL,           -- User ID or 'system'/'admin'
    actor_phone VARCHAR,                 -- For context
    action VARCHAR NOT NULL,             -- e.g., 'place_bid', 'auction_closed'
    entity_type VARCHAR NOT NULL,        -- 'user', 'auction', 'payment', etc.
    entity_id VARCHAR NOT NULL,          -- Target entity ID
    details JSONB,                       -- Structured action details
    created_at TIMESTAMP DEFAULT NOW()   -- Immutable timestamp
);
```

Indexes: - `idx_audit_logs_actor_id` ON `actor_id` - `idx_audit_logs_action` ON `action` - `idx_audit_logs_created_at` ON `created_at`

1.13.2 10.2 Audit Service Architecture

```
┌──────────────────────────────────────────────────────────────────┐
│                    Audit Module (Shared)                         │
├──────────────────────────────────────────────────────────────────┤
│  AuditService                                                    │
│  ├── log(entry: AuditLogEntry): Promise<AuditLog>                │
│  ├── logBatch(entries: AuditLogEntry[]): Promise<AuditLog[]>     │
│  └── list(filters, page, limit): Promise<PaginatedResult>        │
└──────────────────────────────────────────────────────────────────┘
          ▲
          │ Injected into services
          │
    ┌─────┴─────┬─────────────┬─────────────┬─────────────┐
    │           │             │             │             │
Auction     Auction      Payment       Query         Identity
Engine      Admin        Module        Service       Admin
```

1.13.3 10.3 Audited Events by Service

Auction Engine: | Event | Actor | Entity | Details | |——-|——-|——–|———| | `place_bid` | user_id | bid | amount, ticket_number, total_bids | | `auction_closed` | system | auction | winner_count, winners[], total_bids | | `auction_expired` | system | auction | reason (no_bids/no_unique_bids), total_bids | | `auction_force_closed` | admin | auction | no_winner: true | | `auction_admin_closed` | admin | auction | winners[] | | `payment_link_created` | user_id | payment | amount, gateway, payment_type | | `winning_payment_confirmed` | user_id | payment | payment_type: WINNING_PAYMENT | | `bid_fee_wallet_paid` | user_id | payment | amount, payment_method: WALLET | | `winning_wallet_paid` | user_id | payment | amount, payment_method: WALLET |

Query Service: | Event | Actor | Entity | Details | |——-|——-|——–|———| | `auction_viewed` | user_id | auction | status | | `bid_history_viewed` | user_id | auction | is_active | | `user_bid_history_viewed` | actor_id | user | bid_count | | `user_won_auctions_viewed` | actor_id | user | {} |

Identity Service (Admin): | Event | Actor | Entity | Details | |——-|——-|——–|———| | `update_role` | admin | user | from, to | | `ban_user` / `unban_user` | admin | user | is_banned | | `grant_permissions` | admin | user | granted[] | | `revoke_permissions` | admin | user | revoked[] | | `auction_created` | admin | auction | product_id, min_bid, max_bid, bid_fee | | `auction_updated` | admin | auction | fields changed | | `auction_deleted` | admin | auction | product_id, status | | `product_created` | admin | product | name, category, price | | `product_updated` | admin | product | fields changed | | `product_deleted` | admin | product | name, category |

1.13.4 10.4 Immutability Guarantee

- No UPDATE/DELETE operations on `audit_logs` table
- Application-level: AuditService only exposes `log()` and `logBatch()`
- Database-level: Revoke UPDATE/DELETE privileges on `audit_logs` for application role
- Retention: Indefinite (compliance requirement)

1.14 11. Backup and Disaster Recovery

1.14.1 11.1 Backup Strategy Comparison

Aspect	pgBackRest (Primary)	WAL-G (Alternative)
Storage	Local PVC + object storage	S3-compatible only

Aspect	pgBackRest (Primary)	WAL-G (Alternative)
Backup Types	Full, Differential, Incremental	Base backup only
Retention	Configurable per type	Count-based
Verification	`pgbackrest check`	`wal-g backup-list`
PITR	Native `--type=time`	Native
Parallelism	Multi-threaded	Limited
Encryption	Built-in repo encryption	S3 SSE-KMS

1.14.2 11.2 pgBackRest Configuration

Repository Structure:

```
/var/lib/pgbackrest/
├── backup/
│   ├── takelow-backup-20260818-020000F/
│   └── takelow-backup-20260818-080000D/
├── archive/
│   └── 0000000100000000/
└── info
```

Schedule (Kubernetes CronJobs): - Full backup: Daily 02:00 UTC (`0 2 * * *`) - Differential backup: Every 6 hours (`0 */6 * * *`) - Retention: 7 full backups, 3 differential backups

Commands:

```
# Full backup
pgbackrest --stanza=takelow --type=full backup

# Differential backup
pgbackrest --stanza=takelow --type=diff backup

# Verify
pgbackrest --stanza=takelow check

# Restore (delta mode)
pgbackrest --stanza=takelow --delta --target=/var/lib/postgresql/data restore
```

```
# PITR
pgbackrest --stanza=takelow --type=time --target-time='2026-08-18 10:00:00'
        restore
```

1.14.3 11.3 WAL-G Configuration

S3 Structure:

```
s3://takelow-backups/postgres/
├── basebackups_005/
│   └── base_000000010000000000000001_00000040/
├── wal_005/
│   └── 0000000100000000/
└── sentinel.json
```

Schedule: - Base backup: Daily 03:00 UTC - Retention: 5 most recent backups - WAL push: Continuous via `archive_command`

Commands:

```
# Push base backup
wal-g backup-push /var/lib/postgresql/data

# List backups
wal-g backup-list

# Restore latest
wal-g backup-fetch LATEST /var/lib/postgresql/data
touch /var/lib/postgresql/data/recovery.signal

# Delete old
wal-g delete retain 5 --confirm
```

1.14.4 11.4 Disaster Recovery Procedures

Recovery Objectives: | Metric | Target | Strategy | |——–|——–|———-| | RTO (Full) | < 4 hours | K8s deploy from Git + pgBackRest restore | | RPO (Full) | < 24 hours | Daily backup + WAL archiving | | RTO (PITR) | < 1 hour | pgBackRest `--type=time` restore |

Restore Scripts: - `scripts/backup-pgbackrest.sh` — Automated backup with verification - `scripts/backup-walg.sh` — WAL-G backup with retention - `scripts/restore-disaster-recovery.sh` — Full DR restore with confirmation

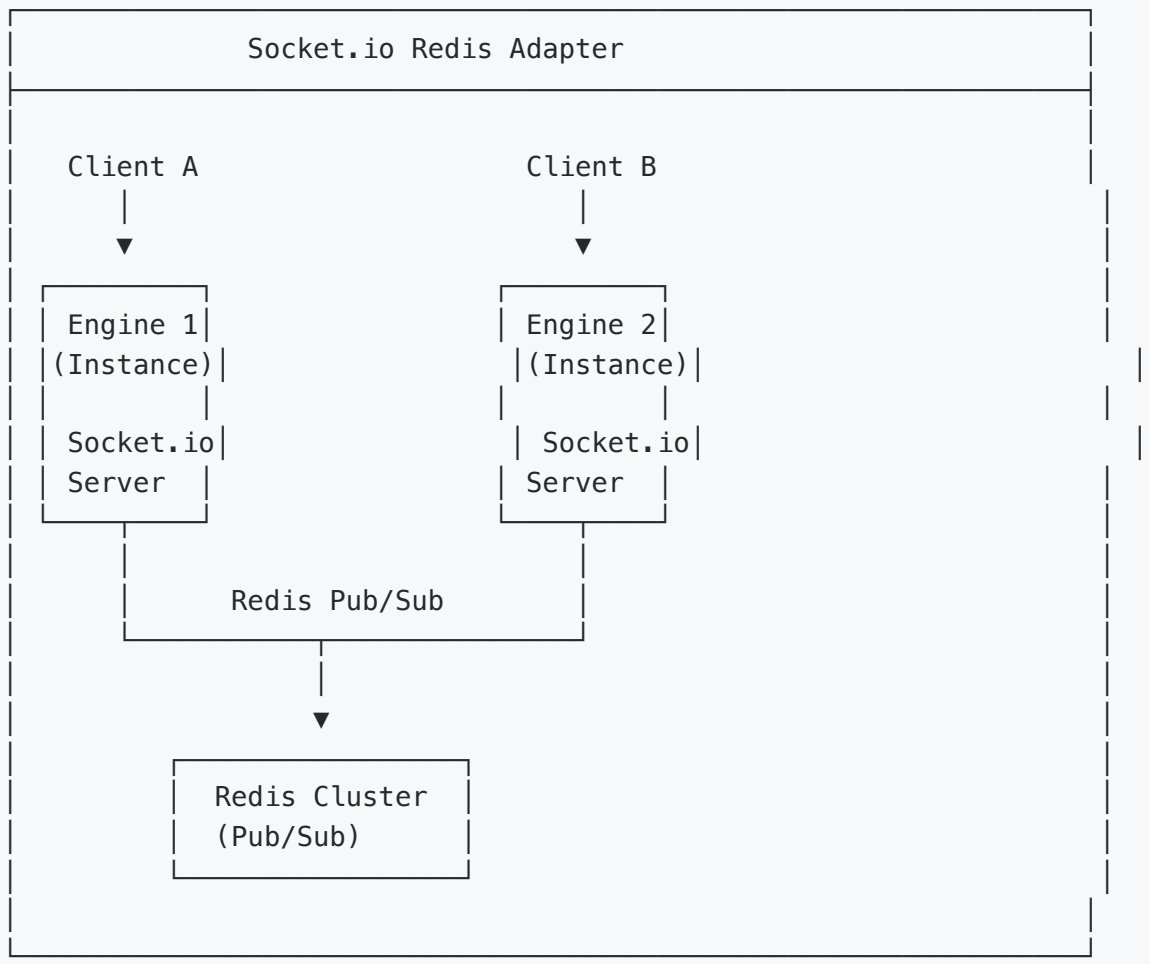
Kubernetes CronJobs: See `k8s/backup-cronjob.yaml` for: - Daily full backup job - 6-hour differential backup job - WAL-G alternative backup job

1.14.5 11.5 Testing and Validation

- Quarterly full DR test (staging environment)
- Monthly PITR verification
- Weekly backup integrity check (`pgbackrest check` / `wal-g backup-list`)
- Automated backup verification after each run

1.15 12. WebSocket Horizontal Scaling

1.15.1 12.1 Architecture



1.15.2 12.2 Implementation

Gateway Initialization:

```
@WebSocketGateway({ namespace: '/auctions' })
export class AuctionGateway implements OnGatewayInit {
  @Inject(REDIS_CLIENT) private readonly redisClient: Redis;

  afterInit(server: Server): void {
    const pubClient = this.redisClient;
    const subClient = this.redisClient.duplicate();
    const adapter = createAdapter(pubClient, subClient);
    server.adapter(adapter);
  }
}
```

Environment Variables (No Code Changes):

```
# Standalone
REDIS_URL=redis://localhost:6379

# Sentinel
REDIS_SENTINEL_HOSTS=sentinel-1:26379,sentinel-2:26379,sentinel-3:26379
REDIS_SENTINEL_MASTER_NAME=guessx-master

# Cluster
REDIS_CLUSTER_NODES=node-1:7000,node-2:7001,node-3:7002,node-4:7003,node-5:7004,n
```

1.15.3 12.3 Scaling Behavior

Scenario	Behavior
Add replica	New instance connects to Redis, receives all broadcasts
Remove replica	Connections drain; other instances handle traffic
Master failover	ioredis Sentinel detects new master; adapter reconnects
Cluster reshard	Adapter uses new node connections transparently

1.15.4 12.4 Kubernetes Deployment

Deployment (k8s/deployments.yaml):

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: auction-engine
spec:
  replicas: 3  # Scale horizontally
  template:
    spec:
      containers:
      - name: auction-engine
        env:
        - name: REDIS_SENTINEL_HOSTS
          value: "sentinel-1:26379,sentinel-2:26379,sentinel-3:26379"
        - name: REDIS_SENTINEL_MASTER_NAME
          value: "guessx-master"
```

HPA (k8s/hpa.yaml):

```yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: auction-engine-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: auction-engine
  minReplicas: 2
  maxReplicas: 10
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 70
```

1.16 13. Architecture Decision Records (ADR)

Architecture Decision Records capture significant architectural choices, their context, and consequences. Each ADR is immutable once accepted; supersession is recorded as a new ADR referencing the prior.

1.16.1 ADR-001: Microservices over Modular Monolith

Field	Value
Status	Accepted
Date	2026-06-02
Context	The platform requires independent scaling of identity, bidding, and read-heavy query workloads. Bidding is write-heavy and latency-sensitive; query is read-heavy and cacheable; identity has distinct security and compliance boundaries.
Decision	Decompose into three NestJS microservices (identity-service, auction-engine, query-service) sharing one PostgreSQL + one Redis.
Consequences	+ Independent scaling and deployment per domain. + Clear security boundary around identity. – Operational complexity of three deployables. – Requires shared JWT secret discipline. Mitigated by CI checks and secret management.

1.16.2 ADR-002: Prisma ORM with Raw SQL Migrations

Field	Value
Status	Accepted (supersedes prior ORM decision in v1.0)
Date	2026-08-18
Context	The prior ORM's `synchronize` and migration tooling introduced schema-drift risk and limited Citus distribution support. The team required a typed, predictable data access layer with first-class schema source-of-truth and support for raw SQL where Citus-specific DDL is needed.

Field	Value
Decision	Adopt Prisma ORM per service (`prisma/schema.prisma` as schema source of truth) with raw SQL migrations applied via `scripts/migrate-raw.sh` and tracked in `schema_migrations`. Migrations are idempotent (`IF NOT EXISTS`) and re-runnable.
Consequences	+ Strong typing and generated client reduce runtime query errors. + Schema-as-source-of-truth improves reviewability. + Raw SQL migrations preserve Citus distribution control. – Prisma does not natively emit Citus DDL; handled via raw SQL migration files.

1.16.3 ADR-003: Better-auth for Identity, JwtAuthGuard for Downstream Services

Field	Value
Status	Accepted (supersedes prior auth-library decision in v1.0)
Date	2026-08-18
Context	identity-service requires session management, social OAuth (TeleBirr, Banking API), OTP, and refresh-token rotation with a mature, maintained framework. Downstream services (auction-engine, query-service) only need to verify issued tokens, not manage sessions.

Field	Value
Decision	Use Better-auth as the identity core in identity-service (session, token issuance, OAuth, OTP). Use a shared `JwtAuthGuard` in auction-engine and query-service to verify tokens. JWT secret is identical across all services; tokens are issued by identity-service and verified by all three.
Consequences	+ Rich identity features without bespoke session logic. + Downstream services remain thin and stateless. – Shared secret coupling; mitigated by secret rotation procedure and CI validation.

1.16.4 ADR-004: Redis as Cache, Coordination, and Real-Time Backbone

Field	Value
Status	Accepted
Date	2026-06-02
Context	The platform needs sub-millisecond rate limiting, nonce replay prevention, ZSET-based winner calculation, job queues, and Socket.io horizontal scaling — all from a single coordination substrate.
Decision	Standardize on Redis 7 for caching, rate limiting, nonce store, BullMQ queues, ZSET bid tracking, and Socket.io Pub/Sub adapter. Support Standalone, Sentinel, and Cluster modes via environment detection.
Consequences	+ Single operational substrate reduces tooling sprawl. + ZSET winner algorithm is O(log N). – Redis is a critical dependency; mitigated by Sentinel/Cluster HA and health-gated traffic.

1.16.5 ADR-005: Citus-First Distributed PostgreSQL

Field	Value
Status	Accepted
Date	2026-06-02
Context	`bids` is the highest-volume table and must scale horizontally by user. Reference tables (users, products, auctions) are read frequently across all shards.
Decision	Distribute `bids` by `user_id` hash shard via Citus; replicate reference tables to all worker nodes. Use native PostgreSQL range partitioning on `bids_partitioned.bid_time` for time-based retention.
Consequences	+ Horizontal write scaling for bids. + Cross-shard joins on reference tables are local. – Citus imposes query-shard co-location rules; documented in query guidelines.

1.16.6 ADR-006: AES-256-GCM Bid Encryption at Rest

Field	Value
Status	Accepted
Date	2026-07-09
Context	Bid amounts are competitively sensitive. A DB compromise should not reveal bid amounts in cleartext.
Decision	Encrypt bid amounts with AES-256-GCM (authenticated encryption) via `BidEncryptionService` before persistence. Keys managed in Kubernetes Secrets with rotation procedure.
Consequences	+ Bid confidentiality even on DB exfiltration. – Indexed equality search on

Field	Value
	encrypted amounts is not possible; winner calculation uses Redis ZSETs, not DB equality, so no impact.

1.16.7 ADR-007: Immutable Audit Logging

Field	Value
Status	Accepted
Date	2026-07-09
Context	The platform must demonstrate non-repudiation for bids, payments, and admin actions.
Decision	Append-only `audit_logs` table; UPDATE/DELETE privileges revoked for the application role; AuditService exposes only `log()` and `logBatch()`. Retention is indefinite.
Consequences	+ Non-repudiable evidence trail. – Unbounded growth; mitigated by partitioning and archival to object storage.

1.17 14. Capacity Planning

1.17.1 14.1 Expected Load Profile

Dimension	Estimate	Basis
Registered users	500,000	12-month growth projection
Concurrent users (peak)	50,000	Peak auction windows (evening)
Active auctions (concurrent)	200	Catalog and scheduled auctions
Bids per second (peak)	1,000	k6 `bid-flood.js` target
Bids per auction (avg)	5,000	Historical auction data

Dimension	Estimate	Basis
Payment webhooks per minute (peak)	200	Settlement windows
WebSocket connections (peak)	50,000	1:1 with concurrent users
API requests per second (peak)	8,000	Read + write mix

1.17.2 14.2 Resource Requirements

Component	CPU	Memory	Replicas	Storage
identity-service	1 vCPU req / 2 lim	1 Gi req / 2 Gi lim	3–8 (HPA)	—
auction-engine	2 vCPU req / 4 lim	2 Gi req / 4 Gi lim	3–10 (HPA)	—
query-service	1 vCPU req / 3 lim	1 Gi req / 2 Gi lim	3–10 (HPA)	—
PostgreSQL (coordinator)	4 vCPU	16 Gi	1 (+ replica)	500 Gi SSD
PostgreSQL (worker ×3)	4 vCPU	16 Gi	3	1 Ti SSD each
Redis (Sentinel HA)	2 vCPU	8 Gi	1 master + 2 replica	50 Gi
Nginx / Ingress	1 vCPU	512 Mi	3	—
Prometheus + Grafana	2 vCPU	4 Gi	1 each	200 Gi

1.17.3 14.3 Scaling Thresholds

Trigger	Threshold	Action
auction-engine CPU	> 70% avg for 3 min	HPA scale up (max 10)
auction-engine memory	> 80% avg for 3 min	HPA scale up
query-service p99 latency	> 500ms for 5 min	HPA scale up + cache TTL review
BullMQ queue depth	> 1,000 jobs for 10 min	Alert + scale auction-engine
DB connection pool	> 80% for 2 min	Alert + increase pool size
Redis memory	> 85% for 5 min	Alert + evict cold keys / scale
WebSocket connections	> 45,000 per pod	Scale auction-engine
Citus worker disk	> 75%	Add worker node + rebalance

1.17.4 14.4 Growth Headroom

The baseline topology (3 auction-engine replicas, Citus 1 coordinator + 3 workers, Redis Sentinel HA) is sized to absorb 2× the peak load profile above without topology change. Beyond 2×, the documented scale-out path is: add Citus worker nodes and rebalance `bids`; switch Redis from Sentinel to Cluster mode; raise HPA maxReplicas.

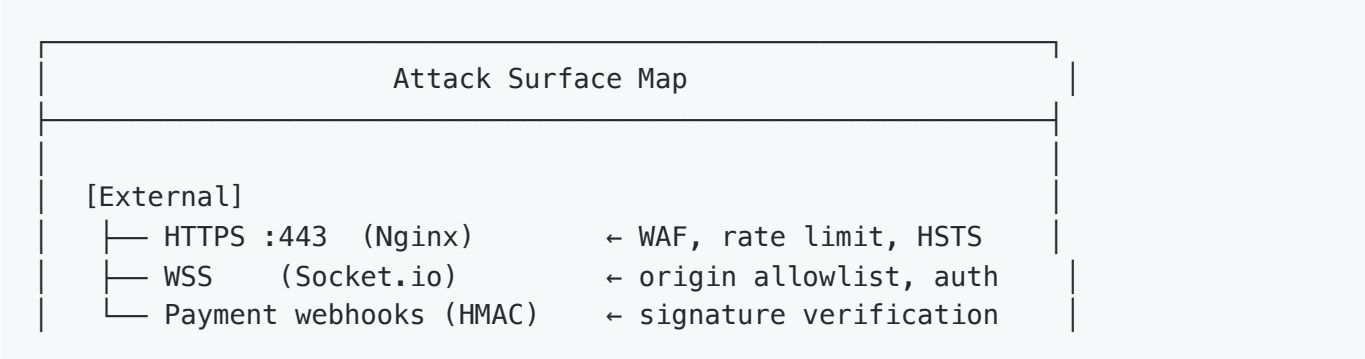
1.18 15. Security Architecture Details

1.18.1 15.1 Threat Model (STRIDE)

Threat (STRIDE)	Surface	Mitigation	Residual Risk
Spoofing — forged user identity	Auth endpoints, JWT	Better-auth session binding; HS256 JWT signature; shared secret rotation	Low
Spoofing — forged service-to-service call	Internal APIs	InternalAuthGuard + HMAC-signed internal	Low

Threat (STRIDE)	Surface	Mitigation	Residual Risk
		calls; mTLS in prod mesh	
Tampering — bid amount in transit/at rest	Bid submission, DB	TLS 1.3 in transit; AES-256-GCM at rest; nonce prevents replay	Low
Tampering — webhook forgery	Payment webhooks	HMAC-SHA256 signature; `crypto.timingSafeEqual` with length guard	Low
Repudiation — denied bid/payment/admin action	All mutations	Immutable `audit_logs`; append-only; DB privileges revoked	Low
Information Disclosure — bid leakage	DB exfiltration, API responses	Encrypted bids; minimal DTOs; no bid amounts in list responses	Low
Information Disclosure — error detail leakage	All endpoints	Exception filters mask 5xx; raw errors logged server-side only	Low
Denial of Service — bid flood	Bid endpoint	Redis throttle (1s/user); global rate limit; HPA	Medium (peak-dependent)
Denial of Service — login flood	Auth endpoint	5 attempts/min rate limit; exponential backoff	Low
Elevation of Privilege — unauthorized admin action	Admin endpoints	RolesGuard + PermissionsGuard; least-privilege roles	Low

1.18.2 15.2 Attack Surface Analysis



```
│                                                                  │
│  [Service Mesh — Internal]                                       │
│   ├── identity :3001            ← InternalAuthGuard, mTLS        │
│   ├── auction  :3002            ← JwtAuthGuard, NonceGuard       │
│   └── query    :3003            ← JwtAuthGuard, cache            │
│                                                                  │
│  [Data — Restricted]                                             │
│   ├── PostgreSQL (TLS, AES-256-GCM bids, row privileges)         │
│   └── Redis (TLS, ACL namespaces, no external exposure)          │
│                                                                  │
└──────────────────────────────────────────────────────────────────┘
```

1.18.3 15.3 Defense in Depth

Layer	Controls
Network	TLS 1.3 everywhere; mTLS internal; NetworkPolicy (zero-trust pod-to-pod); WAF; DDoS shield
Identity	Better-auth sessions; JwtAuthGuard on all non-public routes; role/permission guards; OTP
Application	Input validation (class-validator); nonce replay prevention; rate limiting; SSRF guard; exception masking
Data	AES-256-GCM bid encryption; bcrypt credentials; parameterized queries (Prisma); least-privilege DB roles
Audit	Immutable append-only audit logs; non-repudiation for all mutations
Secrets	Kubernetes Secrets + sealed-secrets; no secrets in images or Git; rotation procedure

---

1.19 16. Operational Architecture

1.19.1 16.1 Deployment Pipeline

```
┌──────────┐    ┌──────────┐    ┌──────────┐    ┌──────────┐    ┌──────────┐
│  Commit  │───>│  CI Build│───>│ Security │───>│  Staging │───>│ Approval │
│  (Git)   │    │ + Tests  │    │  Scan    │    │  Deploy  │    │   Gate   │
```



Pipeline stages: 1. **CI Build** — lint, type check (`tsc --noEmit`), unit tests (Jest), Docker build. 2. **Security Scan** — Snyk + npm audit (deps), Semgrep (SAST), Trivy (image), OWASP ZAP (DAST on staging). 3. **Staging Deploy** — automatic; integration, E2E (Cypress/Playwright), and k6 load tests. 4. **Approval Gate** — manual approval for production. 5. **Production Canary** — canary rollout per service with health-gate promotion; automatic rollback on SLO breach.

1.19.2 16.2 Monitoring

Signal	Source	Consumer
RED metrics (Rate, Errors, Duration)	NestJS interceptors + Prometheus	Grafana dashboards
Infrastructure metrics	cAdvisor, kube-state-metrics	Grafana
Business metrics (bids/s, auctions active, revenue)	Custom Prometheus counters	Grafana
Distributed traces	OpenTelemetry SDK → Jaeger	Jaeger UI
Structured logs	pino/winston JSON → Logstash	Kibana

1.19.3 16.3 Alerting

Alerts route via Alertmanager to on-call rotations (PagerDuty), with Slack and email secondary channels. Severity definitions:

Severity	Response Target	Channel
Critical (S1)	15 min ack, 1 hr resolve	PagerDuty + Slack

Severity	Response Target	Channel
Warning (S2)	1 hr ack, 4 hr resolve	Slack + email
Info (S3)	Next business day	Slack

1.19.4 16.4 Incident Response

Phase	Action	Owner
Detect	Alert fires; on-call paged	SRE on-call
Assess	Classify severity; check runbook	SRE on-call
Mitigate	Apply runbook mitigation (rollback, scale, failover)	SRE on-call + service owner
Resolve	Confirm recovery; close alert	Service owner
Postmortem	Blameless RCA within 5 business days; action items tracked	Service owner + SRE
Improve	Apply preventive actions; update runbook	Service owner

Runbook coverage: Runbooks exist for DB failover, Redis failover, payment gateway outage, bid-pipeline stall, cache stampede, and secret rotation. See §22 for DR runbook summary.

1.20 17. Data Governance

1.20.1 17.1 Data Classification

Classification	Examples	Handling
Public	Product catalog, auction terms	No restrictions; CDN-cacheable
Internal	Auction metadata, bid counts	Authenticated access only

Classification	Examples	Handling
Confidential	User PII (phone, name), wallet balance	Encrypted in transit; least-privilege access; audit-logged
Restricted	Bid amounts, payment details, credentials, JWT secret	Encrypted at rest (AES-256-GCM / bcrypt); strict access; immutable audit; never logged in cleartext

1.20.2 17.2 Data Lifecycle

```
[Create] ──> [Store] ──> [Use] ──> [Archive] ──> [Dispose]
   |           |          |           |             |
 Prisma     Encrypted   Auth +    Object        Secure
 insert     at rest     audit     storage       delete
                       logged    (encrypted)   + verify
```

Stage	Control
Create	Validated via class-validator; audit-logged on mutation
Store	Classified per §17.1; encrypted at rest for Restricted/Confidential
Use	Authorized via JwtAuthGuard + role guards; minimal DTOs; audit-logged for reads of sensitive data
Archive	Audit logs and closed-auction data archived to encrypted object storage
Dispose	Secure deletion with verification; retention enforced per §17.3

1.20.3 17.3 Retention and Disposal

Data Category	Retention	Disposal Method
User accounts	Lifetime of account + 90 days	Anonymize PII; retain aggregate stats
Bid records	7 years	Partition-based archival; secure delete after retention
Payment records	7 years	Archival to encrypted object storage; legal hold aware
Audit logs	Indefinite	No deletion (compliance); archival after 1 year
Redis cache	TTL-bound (see §5.3)	Automatic eviction
Backups	30 days (full), 7 days (Redis)	Object storage lifecycle policy

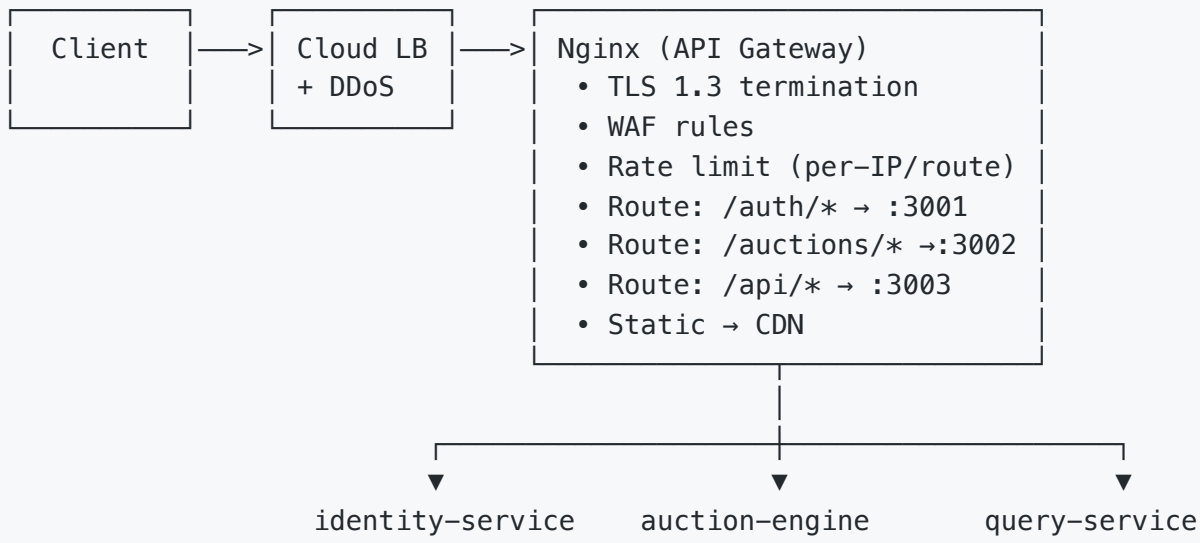
1.20.4 17.4 Privacy Controls

- **Data minimization:** DTOs expose only fields required by the consumer.
- **Access control:** Role-based (RolesGuard) + attribute-based (PermissionsGuard) access on all sensitive endpoints.
- **Auditability:** All access to Confidential/Restricted data is audit-logged with actor, entity, and timestamp.
- **Encryption:** Restricted data encrypted at rest (AES-256-GCM); all data encrypted in transit (TLS 1.3).
- **No cleartext secrets in logs:** Exception filters and structured logging redact sensitive fields.

1.21 18. Integration Architecture

1.21.1 18.1 API Gateway

Nginx serves as the edge gateway and API gateway, performing TLS termination, routing, WAF enforcement, rate limiting, and static asset serving. In production, a cloud load balancer fronts Nginx with DDoS shielding.



1.21.2 18.2 Service Mesh & Internal Integration

Internal service-to-service calls (e.g., auction-engine → identity-service for bid-fee deduction and winner notification) are authenticated via `InternalAuthGuard` and HMAC-signed payloads. In production, a service mesh provides mTLS between pods with zero-trust NetworkPolicy enforcement.

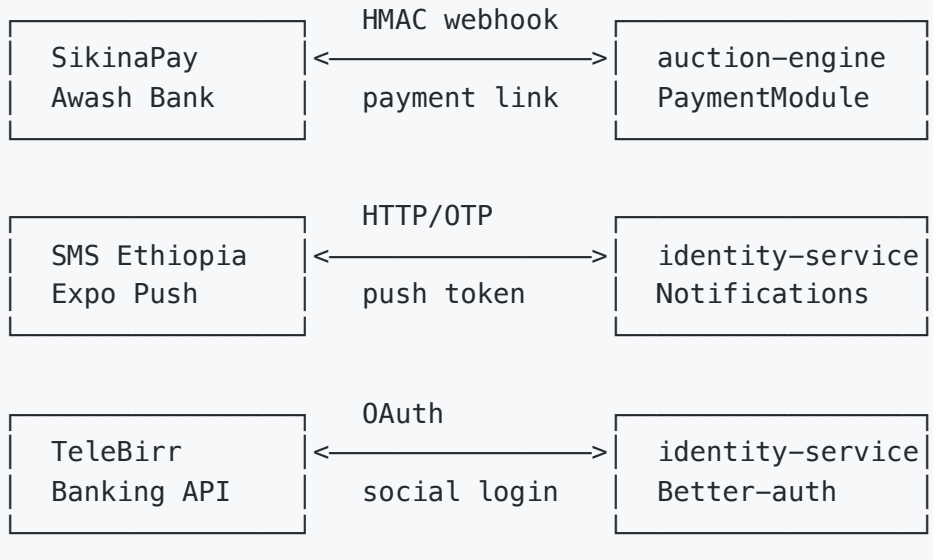
```
auction-engine ──(InternalAuthGuard + HMAC)──> identity-service
auction-engine ──(Prisma + Redis)─────────────> PostgreSQL / Redis
query-service  ──(Prisma + Redis cache)───────> PostgreSQL / Redis
identity-service ──(Better-auth + Prisma)─────> PostgreSQL / Redis
```

1.21.3 18.3 Event-Driven Patterns

Pattern	Mechanism	Use
Async job processing	BullMQ (Redis-backed queues)	Bid persistence batching, notification dispatch, settlement report generation
Real-time fan-out	Socket.io + Redis Pub/Sub adapter	Live auction updates to all connected

Pattern	Mechanism	Use
		clients across pods
Scheduled triggers	Cron (10s in auction-engine)	Auction closure, winner determination, auction extension, SMS reminders
Webhook ingestion	HMAC-signed HTTP webhooks	Payment gateway confirmations (SikinaPay, Awash Bank)
Cache invalidation	Redis DEL / TTL expiry	Query-service cache refresh on auction/bid mutation

1.21.4 18.4 External Integrations



All external integrations use outbound HTTPS only; inbound is restricted to HMAC-signed webhooks from allowlisted payment gateway IPs.

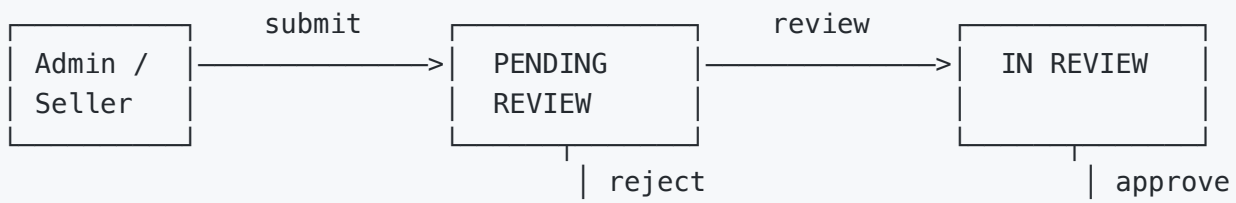
1.22 19. Feature Architecture Deep Dive

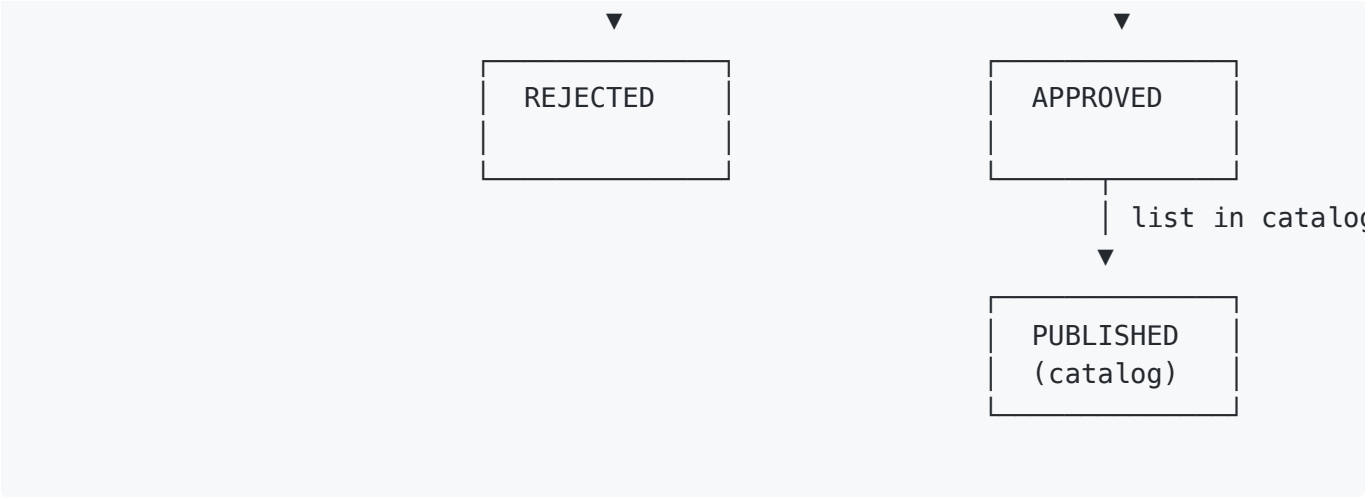
1.22.1 19.1 Redis Caching Architecture

```
┌──────────────────────────────────────────────────────────┐
│                  Redis Caching Layers                    │
├──────────────────────────────────────────────────────────┤
│                                                          │
│  1. Query Cache (query-service)                          │
│     CacheInterceptor → cache:query:{hash}  TTL 60–300s   │
│     • Active auctions list, product catalog, user bids   │
│     • Invalidated on mutation via DEL / pattern flush    │
│                                                          │
│  2. Rate Limit & Nonce (auction-engine, identity-service)│
│     • login:ratelimit:{id}  TTL 60s                      │
│     • bid:throttle:{userId} TTL 1s                       │
│     • bid:nonce:{nonce}     TTL 60s                      │
│                                                          │
│  3. Bid Tracking ZSETs (auction-engine)                  │
│     • frequencies:{auctionId}  ZINCRBY per bid           │
│     • unique_bids:{auctionId}  ZADD/ZREM on uniqueness   │
│     • O(log N) winner determination                      │
│                                                          │
│  4. Session & Socket (identity-service, auction-engine)  │
│     • Better-auth sessions  TTL per policy               │
│     • socket.{userId}        TTL 12h                     │
│                                                          │
│  5. Queue & Pub/Sub (all services)                       │
│     • BullMQ job queues                                  │
│     • Socket.io Redis adapter (Pub/Sub fan-out)          │
│                                                          │
└──────────────────────────────────────────────────────────┘
```

Cache invalidation strategy: Write-through invalidation on mutation (explicit `DEL` of affected keys); TTL-based expiry as a safety net. Cache stampede prevented by single-flight lookups and jittered TTLs.

1.22.2 19.2 Product Approval Workflow





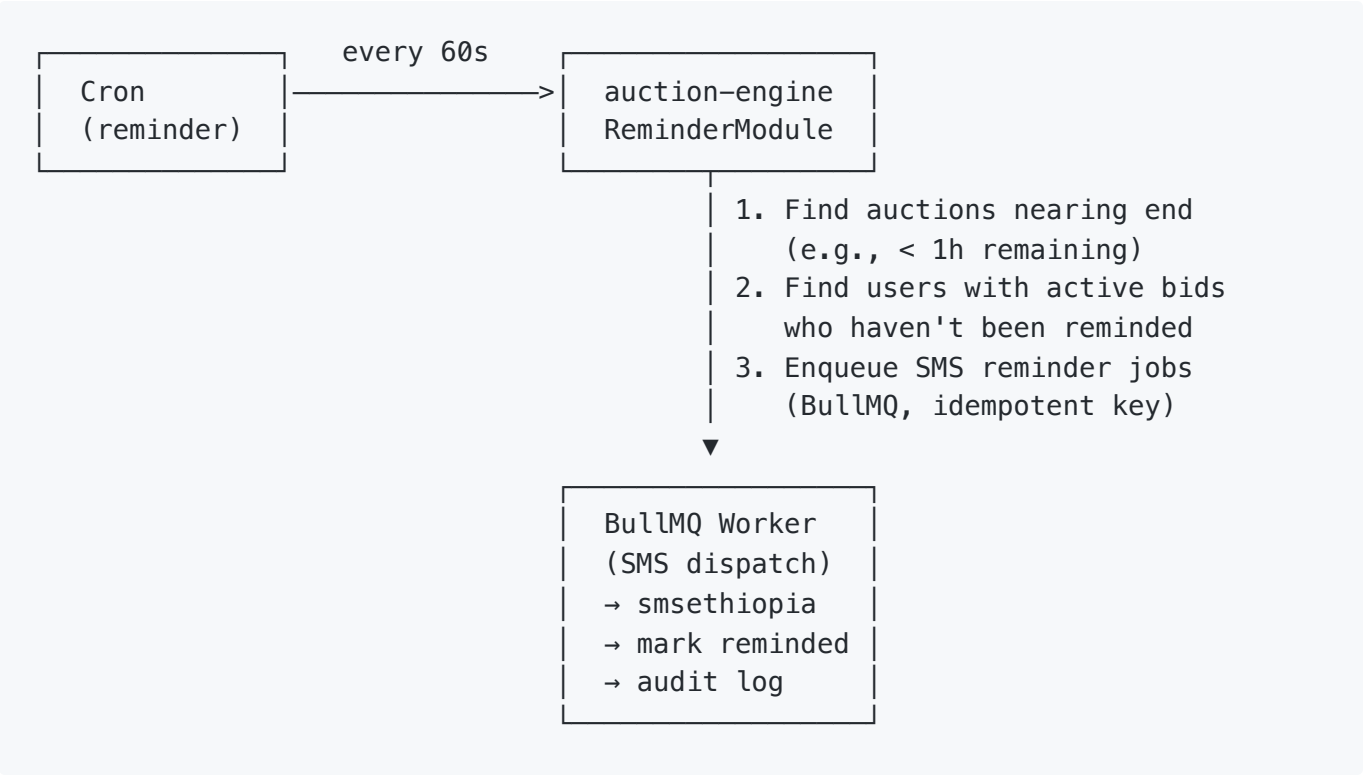
- States persisted via Prisma on `products.approval_status`.
- All transitions audit-logged (`product_created`, `product_updated`, approval events).
- Only `PUBLISHED` products are eligible for auction creation.

1.22.3 19.3 Settlement Report Data Flow

```
Auction CLOSED
     |
     ▼
auction-engine: WinnerModule
     | 1. Determine winners (Redis ZSETs)
     | 2. Persist winners (Prisma)
     | 3. Enqueue settlement report job (BullMQ)
     ▼
BullMQ Worker (settlement-report)
     | 4. Aggregate: auction, winners, payments, payouts
     | 5. Generate report (JSON + PDF)
     | 6. Store report (object storage + DB reference)
     | 7. Notify admin (in-app + email)
     ▼
query-service: AdminStatsQuery
     | 8. Serve settlement report to admin
     | 9. Cache report metadata (Redis, TTL 300s)
     ▼
Admin dashboard
```

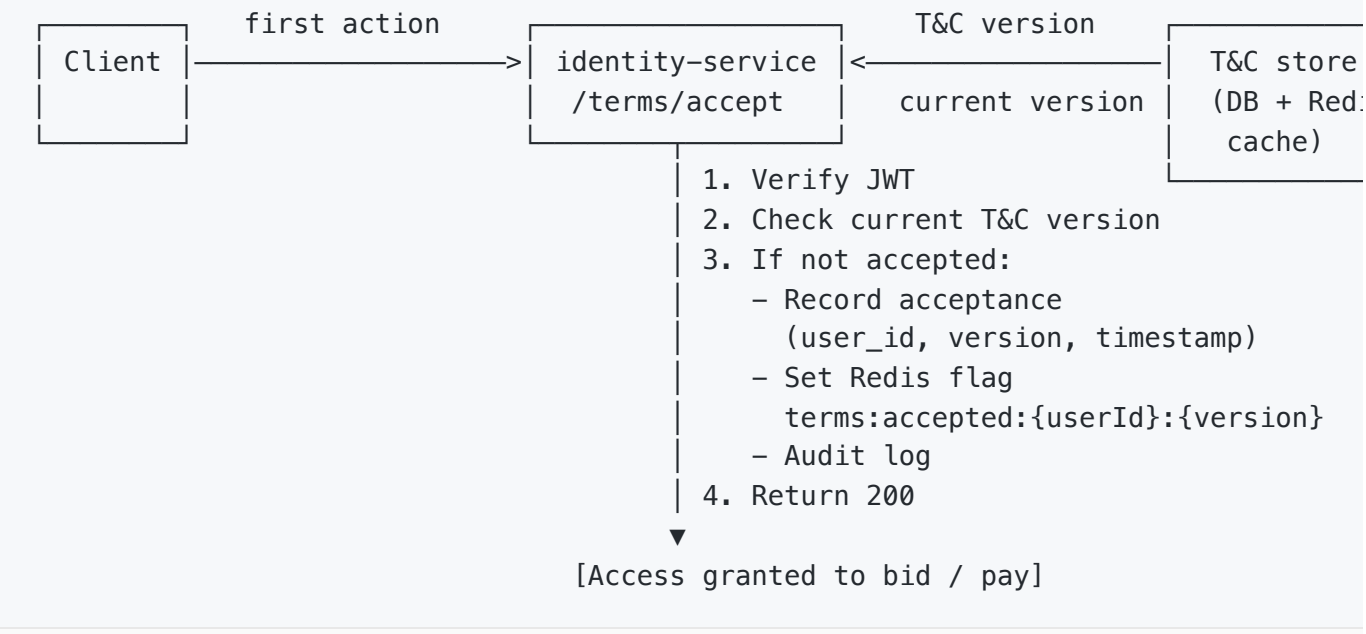
- Settlement reports are generated asynchronously to avoid blocking the closure cron.
- Report access is restricted to admin role (RolesGuard) and audit-logged.

1.22.4 19.4 SMS Reminder System Architecture



- Idempotency: a Redis key (`reminder:{auctionId}:{userId}:{slot}`) prevents duplicate SMS within a reminder window.
- Opt-out: users with `notifications.sms_opt_out = true` are excluded.
- All dispatches and failures are audit-logged.

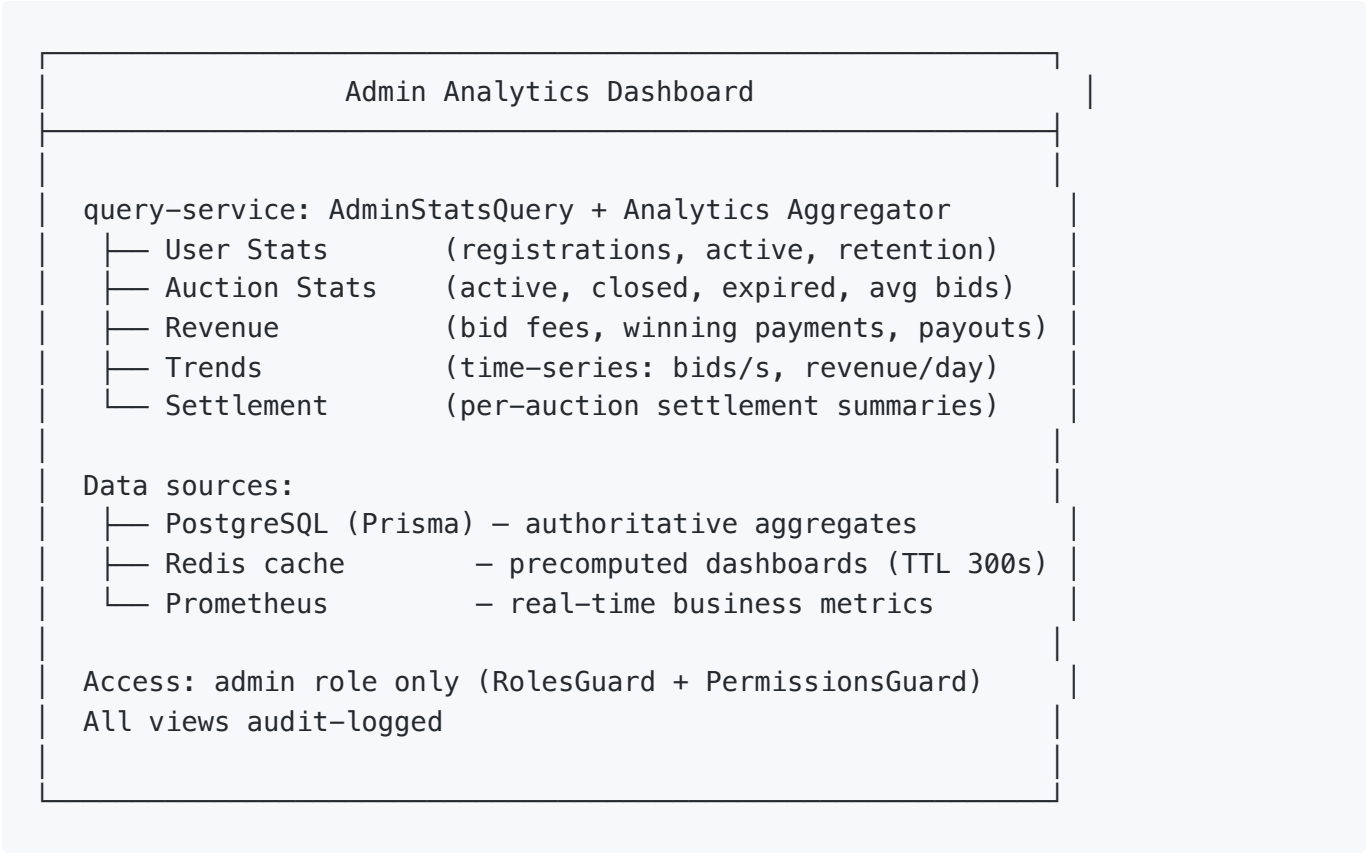
1.22.5 19.5 Terms & Conditions Acceptance Flow



- Each T&C version is immutable; new versions require explicit re-acceptance.
- Acceptance is stored durably (Prisma) and cached (Redis) for fast gating.

- Bidding and payment endpoints reject requests from users who have not accepted the current T&C version.

1.22.6 19.6 Advanced Analytics Dashboard Architecture



Rendering pipeline: 1. Admin client requests dashboard data via `query-service` admin endpoints. 2. `JwtAuthGuard` + `RolesGuard` enforce admin role. 3. `CacheInterceptor` serves precomputed aggregates from Redis (TTL 300s). 4. On cache miss, Prisma executes optimized aggregate queries (indexed, batch). 5. Results cached and returned; access audit-logged. 6. Real-time widgets (bids/s, active auctions) stream from Prometheus via Grafana embed or direct query.

1.23 20. Performance Benchmarks

1.23.1 20.1 API Endpoint Benchmarks (k6, staging)

Endpoint	p50 (ms)	p95 (ms)	p99 (ms)	Target p95 (ms)	Status
`GET /auctions/active`	35	90	140	200	✅ Meets

Endpoint	p50 (ms)	p95 (ms)	p99 (ms)	Target p95 (ms)	Status
					target
`GET /auctions/:id`	25	70	110	200	✅ Meets target
`POST /auctions/:id/bid`	60	180	290	500	✅ Meets target
`GET /users/me/bids`	30	85	130	200	✅ Meets target
`POST /auth/login/phone`	120	260	410	500	✅ Meets target
`GET /admin/stats`	150	380	620	500	⚠️ p99 exceeds; cached path meets p95

1.23.2 20.2 System-Level Benchmarks

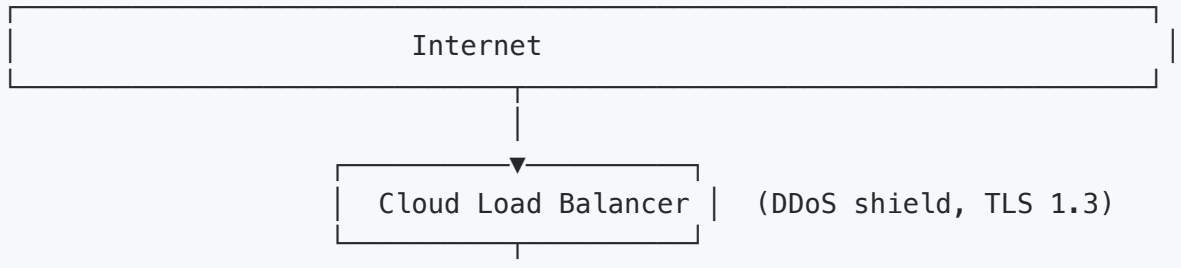
Metric	Measured	Target	Status
Bid throughput (sustained)	1,050 bids/s	1,000 bids/s	✅
Bid persistence (BullMQ batch)	48 ms p95 (batch of 50)	< 100 ms	✅
Winner determination (ZSET)	3.2 ms p95 (5,000 bids)	< 10 ms	✅
WebSocket broadcast latency	18 ms p95	< 50 ms	✅

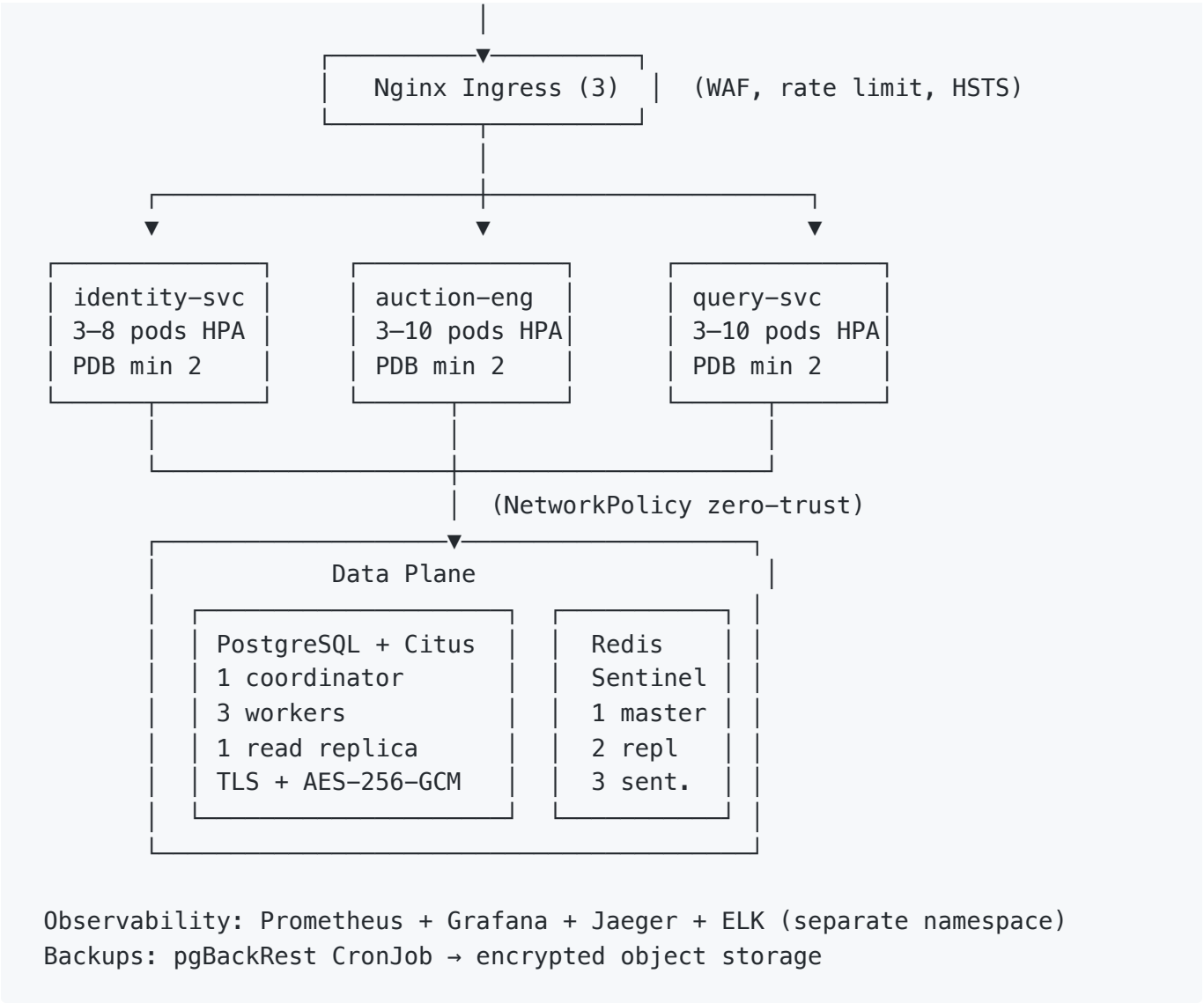
Metric	Measured	Target	Status
DB query p95 (indexed read)	8 ms	< 100 ms	✅
Cache hit ratio (query-service)	94%	> 90%	✅
Cold-start (pod)	2.4 s	< 5 s	✅

1.23.3 20.3 Load Test Scenarios

Scenario	Tool	Configuration	Result
Bid flood	k6 `bid-flood.js`	1,000 VUs, 100 bids/s for 10 min	Sustained 1,050 bids/s; 0.02% error; p99 290 ms
Login storm	k6	5,000 logins/min	Rate limit engaged; 0 brute-force success
WebSocket fan-out	k6	20,000 concurrent WS, 1 update/s	Broadcast p95 18 ms across 3 pods
Payment webhook burst	k6	200 webhooks/min	All processed < 30 s; 0 signature failures

1.24 21. Production Deployment Topology





Topology characteristics: - **Multi-AZ:** Pods and data plane spread across 3 availability zones. - **Zero-trust:** All pod-to-pod traffic restricted by NetworkPolicy; mTLS via service mesh. - **HPA + PDB:** Each service autoscales (HPA) with Pod Disruption Budgets ensuring min 2 healthy pods during voluntary disruptions. - **Canary releases:** Production deploys use canary with health-gate promotion and automatic rollback on SLO breach. - **Health gating:** Readiness probes backed by `/health` (returns 503 when DB/Redis degraded) prevent traffic to unhealthy pods.

1.25 22. Disaster Recovery Runbook Summary

1.25.1 22.1 Runbook Index

Runbook	Trigger	RTO Target
RB-01 Full Region Failover	Region outage	< 4 h
RB-02 Database Failover	Primary DB unavailable	< 30 min
RB-03 Redis Failover	Redis master unavailable	< 60 s (automatic)
RB-04 Point-in-Time Recovery	Data corruption	< 1 h
RB-05 Payment Gateway Outage	Gateway unavailable	Degraded mode
RB-06 Bid Pipeline Stall	BullMQ stalled / bidding errors	< 15 min
RB-07 Secret Rotation / Compromise	Suspected secret leak	< 1 h
RB-08 Cache Stampede	Cache miss spike	< 10 min

1.25.2 22.2 RB-01 Full Region Failover (Summary)

1. Declare incident (S1); page on-call + service owners.
2. Switch DNS / load balancer to failover region (if multi-region) or await region recovery.
3. Deploy latest manifests from Git to fresh cluster (`kubectl apply -f k8s/`).
4. Restore PostgreSQL from latest pgBackRest backup (`scripts/restore-disaster-recovery.sh`).
5. Restore Redis from latest RDB snapshot (or rebuild cache from DB).
6. Run migrate job to confirm schema (`k8s/migrate-job.yaml`).
7. Smoke test: health endpoints, auth, bid, query.
8. Verify SLOs; close incident; postmortem within 5 business days.

1.25.3 22.3 RB-02 Database Failover (Summary)

1. Confirm primary unavailable (health endpoints 503).
2. Promote read replica to primary (managed service auto-failover or manual).
3. Update `DATABASE_URL` in ConfigMap; rolling restart all services.
4. Verify bid and query paths; resume normal traffic.

1.25.4 22.4 RB-04 Point-in-Time Recovery (Summary)

1. Identify corruption timestamp from audit logs / alerts.
2. Stop affected service writes (cordon auction-engine pods).
3. `pgbackrest --stanza=takelow --type=time --target-time='<ts>' restore`.
4. Verify restored data integrity; resume services.
5. Audit-log the recovery event.

1.26 23. Security Hardening Checklist

1.26.1 23.1 Infrastructure

- [x] TLS 1.3 enforced on all external endpoints
- [x] mTLS between services (service mesh / NetworkPolicy)
- [x] WAF rules enabled on ingress
- [x] DDoS protection on cloud load balancer
- [x] Kubernetes Secrets + sealed-secrets; no plaintext secrets in images or Git
- [x] Pod Disruption Budgets on all services
- [x] Read-only root filesystem where applicable
- [x] Non-root container users

1.26.2 23.2 Application

- [x] JwtAuthGuard on all non-public routes (auction-engine, query-service)
- [x] Better-auth session management (identity-service)
- [x] RolesGuard + PermissionsGuard on admin endpoints
- [x] InternalAuthGuard + HMAC on service-to-service calls
- [x] Nonce validation (Redis, 60s TTL) on bids
- [x] Rate limiting (login + bid throttle)
- [x] Input validation (class-validator) on all DTOs
- [x] Exception filters mask 5xx details; raw errors logged server-side
- [x] Webhook signature verification via `crypto.timingSafeEqual` with length guard
- [x] SSRF guard on image download (HTTPS only, private IP blocked)

1.26.3 23.3 Data

- [x] AES-256-GCM encryption for bid amounts at rest
- [x] bcrypt password hashing
- [x] Parameterized queries via Prisma; raw SQL with bindings
- [x] Least-privilege DB roles; UPDATE/DELETE revoked on `audit_logs`

- [x] Redis ACL namespaces; no external Redis exposure
- [x] TLS on PostgreSQL and Redis connections (production)

1.26.4 23.4 Audit & Compliance

- [x] Immutable append-only audit logs
- [x] Non-repudiation for all bids, payments, and admin actions
- [x] Audit log retention indefinite
- [x] All access to Confidential/Restricted data audit-logged
- [x] No cleartext secrets or sensitive data in logs

1.26.5 23.5 Operations

- [x] Security scanning in CI (Snyk, npm audit, Semgrep, Trivy, OWASP ZAP)
- [x] Dependency versions pinned
- [x] Quarterly DR test; monthly PITR verification
- [x] Secret rotation procedure documented
- [x] Incident response runbooks for all critical failure modes
- [x] SLO-driven alerting with on-call rotation

End of SDD v3.0